Projects (MATH0109)

Richard Hill and John Talbot

March 3, 2024

Contents

1 Introduction **3**

2 Algebra (`Algebra`) **4**

2.1 Central group extensions (`CentralGroupExtensions`) 4

2.2 Dual Numbers (`DualNumbers`) 5

2.3 Free abelian groups (`FreeAbelian`) 6

2.4 Galois Theory (`Galois`) 7

2.5 Semi-direct products (`SemiDirectProduct`) 8

3 Analysis (`Analysis`) **9**

3.1 Accelerated convergence (`Accel`) 9

3.2 Analysis 1 (`Analysis1`) 11

3.2.1 Sequences (`Sequences`) 11

3.2.2 Supremum and Sets (`Sets`) 12

3.2.3 Continuity (`Continuity`) 12

3.2.4 Differentiation (`Differentiation`) 12

3.2.5 Series (`Series`) 14

3.3 Cantor's intersection theorem (`CantorIntersection`) 17

3.4 Cantor Set (`CantorSet`) 19

3.5 Contraction maps on $(\mathbb{N}, d_2)$ (`ContractionMap`) 21

3.6 Dedekind cuts (`Dedekind`) 23

3.7 Set of continuity (`GdeltaCts`) 25

3.8 Fractal convergence (`IFS`) 27

3.9 The intermediate value theorem (`IVT`) 30

3.10 Measure Theory (`MeasureTheory`) 32

3.10.1 Vitali Set (`VitaliSet`) 32

3.11 Topology (`Topology`) 34
3.11.1 Alexandrov topology and orders (`Alexandrov`) 34
3.12 Uniform convergence boundedness and continuity (`Uniform`) . . . 37

4 Combinatorics (`Combinatorics`) 38
4.1 Intersecting Families of sets (`Intersecting`) 38
4.2 Ramsey Theory (`Ramsey`) 40
4.2.1 Ramsey's theorem for two colours (`Graph2Col`) 40
4.2.2 Ramsey's theorem for k colours (`KColourRamsey`) 42
4.2.3 Schur's theorem (`Schur`) 44
4.2.4 Ramsey's theorem (hypergraph version) (`RGraphKCol`) . . 45

5 Linear Algebra (`LinearAlgebra`) 47
5.1 Steinitz Exchange Lemma (`SteinitzExchange`) 47
5.2 Elementary matrices and row operations (`ElemMatRow`) 49
5.3 Gaussian Elimination (`GaussElim`) 50
5.3.1 Reduction to upper triangular form (`GaussStep`) 52
5.3.2 Inverting an invertible matrix (`Inverse`) 54
5.3.3 Non-invertible (`NotInvertible`) 56

6 Number Theory (`NumberTheory`) 58
6.1 Convergence of continued fractions (`ContinuedFrac`) 58
6.2 Divisors (`Divisors`) 59
6.3 The Eisenstein integers (`EisensteinIntegers`) 60
6.4 Infinitude of primes (`Furstenberg`) 62
6.5 Units in the Ring $\mathbb{Z}[\frac{1+\sqrt{5}}{2}]$ (`QuadraticRingUnits`) 64
6.6 Sum of reciprocals of primes (`SumRecipPrimes`) 66
6.7 A Thue equation (`ThueEquation`) 68

1 Introduction

This document describes the end of module projects for MATH0109 Theorem Proving in Lean. For each project there is a page or so of description of the mathematics and some of the Lean code. Each project also has an associated Lean file for you to work on.

You should choose one project and submit your edited version of the project file via Moodle. (See Moodle for details of deadline etc.)

The code for the projects is in the directory `UCLMATH0109/Projects/` where it is arranged by area of mathematics and title.

For example, the code for the Analysis project on the Cantor set can be found in: `UCLMATH0109/Projects/Analysis/CantorSet`

These projects are open-ended and vary in difficulty. Most projects have suggested extensions, and you may submit either more or less than the original outline. We encourage students to tackle something challenging, even if that means leaving parts of it incomplete.

2 Algebra (`Algebra`)

2.1 Central group extensions (`CentralGroupExtensions`)

This is a more advanced topic in group theory.

Let G and K be groups and assume that K is abelian. A cocycle on G with values in K is a function

$$\sigma : G \times G \to K,$$

satisfying the following relation for all $x, y \in G$:

$$\sigma(x, y)\sigma(xy, z) = \sigma(x, yz)\sigma(y, z).$$

If we choose such a σ, then we may define a new group $\tilde{G}_\sigma$, whose elements are pairs (g, k) with $g \in G$ and $k \in K$, where multiplication is defined by

$$(g, k)(g', k') = (gg', kk'\sigma(g, g')).$$

There is a surjective homomorphism $\pi : \tilde{G} \to G$, taking (g, k) to g. The kernel of π is contained in the centre of $\tilde{G}$ (the centre of a group is the set of elements which commute with all other elements of the group), and is isomorphic to K.

Conversely, suppose we have a group $\tilde{G}$ and a surjective homomorphism $\tilde{G} \to G$, whose kernel is isomorphic to K and is in the centre of $\tilde{G}$. Then there is a cocycle σ and an isomorphism $\tilde{G} \cong \tilde{G}_\sigma$.

If $\tau : G \to K$ is any function, then the *coboundary* of τ is the function

$$\partial\tau(x, y) = \tau(xy)\tau(x)^{-1}\tau(y)^{-1}.$$

The coboundary $\partial\tau$ is a cocycle. The group $\tilde{G}_{\partial\tau}$ corresponding to a coboundary is isomorphic to the direct sum of G and K.

The cocycles form an abelian group $Z^2(G, K)$ with the operation of pointwise multiplication. The coboundaries form a subgroups $B^2(G, K)$. If σ and σ' are in the same coset of $B^2(G, K)$ then the groups $\tilde{G}_\sigma$ and $\tilde{G}_{\sigma'}$ are isomorphic.

Prove as much of this as you can in lean. There is an outline of part of this theory in the lean file.

2.2 Dual Numbers (`DualNumbers`)

A ‘dual number‘ is something of the form $a + b \cdot dx$ with $a, b \in \mathbb{R}$. It's clear how to add and subtract two dual numbers. By setting $dx^2 = 0$, we obtain the following definition of multiplication

$$(a + b \cdot dx)(a' + b' \cdot dx) = aa' + (ab' + ba')dx.$$

Define the dual numbers as a commutative ring, and also a vector space over $\mathbb{R}$. Prove that $x+y \cdot dx$ is invertible if and only if $x \neq 0$, and define a function taking an element to its inverse. Show that the dual numbers form an $\mathbb{R}$-algebra, with the obvious operation of scalar multiplication of dual numbers by real numbers.

Define an ordering $a + bdx < a' + b'dx$ iff $a < a'$ or $a = a'$ and $b < b'$. Prove that this makes the dual numbers into an ordered ring (i.e. prove the axioms of an ordered ring, eg. products of positive elements are positive). Prove that dx between 0 and any positive real number.

For any polynomial f, prove that $f(a + dx) = f(a) + f'(a)dx$.

Possible extensions: classify the algebra automorphisms of the dual numbers. Classify the ring homomorphisms between the dual numbers and a field (in either direction).

2.3 Free abelian groups (`FreeAbelian`)

The project is a continuation of problem sheet 6, and concerns additive commutative groups.

Lemma. *If G is a non-zero subgroup of $\mathbb{Z}$, then G is isomorphic to $\mathbb{Z}$.*

Proof. Since G is non-zero, it must contain a non-zero element x. Replacing x by $-x$ if necessary, we see that G must contain a positive integer. Let n be the smallest positive integer in G.

Since G is closed under addition and subtraction, all integers of the form nx are in G for $x \in \mathbb{Z}$. In fact these are all the elements of G. To see why this is true, suppose $g \in G$. Dividing with remainder, we see that there exist integers q, r such that $g = qn + r$ and $0 \leq r < g$. Since g and qn are in G, it follows that $r \in G$, so our choice of n forces r to be zero. Hence $g = nx$.

Since $G = \{nx : x \in \mathbb{Z}\}$, there is an isomorphism $G \cong \mathbb{Z}$, which takes nx to n. $\square$

If you'd like to take this further, then you could try defining the rank of a free abelian group, and proving that the rank is well-defined (i.e. prove that $\mathbb{Z}^n$ is not isomorphic to $\mathbb{Z}^m$ unless $n = m$).

Theorem. *Every subgroup of a free abelian group is a free abelian group.*

Proof. We've proved in sheet 6 that free abelian groups (as defined there) are isomorphic to $\mathbb{Z}^d$.

Let G be a subgroup of $\mathbb{Z}^d$. We'll prove by induction on d that G is a free abelian group. The case $d = 0$ is trivial, so assume $d \geq 0$. There is a homomorphism $\phi : G \to \mathbb{Z}$, which takes every element of G to its d-th coordinate.

Case 1. If $\phi(g) = 0$ for all $g \in G$, then G is (isomorphic to) a subgroup of $\mathbb{Z}^{d-1}$. By the inductive hypothesis G is a free abelian group.

Case 2. If $\phi(g) \neq 0$ for some $g \in G$, then the image of ϕ is a non-zero subgroup H of $\mathbb{Z}$ and the kernel K of ϕ is a subgroup of $\mathbb{Z}^{d-1}$. The previous lemma implies that H is isomorphic to $\mathbb{Z}$. Composing ϕ with this isomorphism, we get a surjective homomorphism $\phi' : G \to \mathbb{Z}$, whose kernel is also K.

By a lemma in sheet 6, there is an isomorphism $G \equiv K \times \mathbb{Z}$. The inductive hypothesis implies that K is a free abelian group. Hence G is a free abelian group. $\square$

2.4 Galois Theory (`Galois`)

Let $\mathbb{F}$ be a field. We'll write Gal $\mathbb{F}$ for the group of field automorphisms of $\mathbb{F}$.

Prove that Gal $\mathbb{F}$ is a group with the operation of composition.

Prove that Gal $\mathbb{Q}$ is the trivial group. To do this, suppose σ is an automorphism of $\mathbb{Q}$. First prove by induction that $\sigma(n) = n$ for all $n \in \mathbb{N}$. Then prove the same equation for $n \in \mathbb{Z}$ and finally for $n \in \mathbb{Q}$ using the field automorphism properties of σ.

Prove that Gal $\mathbb{R}$ is the trivial group. For this, first prove that $\sigma(x) = x$ for all $x \in \mathbb{Q}$ by the same method as above. Then prove that $x \geq 0$ iff $\sigma(x) \geq 0$; this uses the fact that $x \geq 0$ iff x has a square root in $\mathbb{R}$. Finally, prove that $\sigma(x) = x$ for al real x: if this were not true, then there would be a rational number y between x and $\sigma(x)$, and from this we can get a contradiction since $x - y$ would have the opposite sign to $\sigma(x - y)$.

Prove that complex conjugation is a non-trivial element of Gal $\mathbb{C}$. Define what it means for an element of Gal $\mathbb{C}$ to be continuous. Prove that every continuous element of Gal $\mathbb{C}$ is either the identity or complex conjugation.

Possible extensions: choose your own example, (eg. show that the Frobenius element generates the Galois group of a finite field).

2.5 Semi-direct products (`SemiDirectProduct`)

This is a topic from Algebra 4 (Groups and Rings).

- Prove that the set $Aut\ G$ of automorphisms of a group form a group under the operation of composition.
- For any $g \in G$, define the inner automorphism $x \mapsto gxg^{-1}$ as an element of this group. Prove that the set of inner automorphisms of G is a normal subgroup of $Aut\ G$.
- Given a homomorphism $\phi : H \to Aut\ G$, define the semi-direct product $G \rtimes_\phi H$ as a group.
- Call ϕ and ψ equivalent if $\phi = \psi \circ p$ for some automorphism p of H. Prove that this relation is an equivalence relation.
- Prove that if ϕ and ψ are equivalent, then $G \rtimes_\phi H$ is isomorphic to $G \rtimes_\psi H$.

Possible extension: choose two small groups, and determine all of their semi-direct products up to isomorphism.

3 Analysis (`Analysis`)

3.1 Accelerated convergence (`Accel`)

The alternating series test easily proves that the following series converges (conditionally)

$$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}.$$

In fact, as proved by Leibniz, it converges to $\pi/4$.

Suppose we want to approximate $\pi/4$ using this series, how many terms would we need to take?

For any convergent alternating series with decreasing terms such as this, the error after n terms is roughly the nth term, so in this case $\sim 1/(2n+1)$. So if we wanted to compute $\pi/4$ to 100 decimal places we would require around 10^{100} terms.

In this project we will consider a technique due to Euler for accelerating the convergence of a series such as this; introducing a new series that converges to the same limit, but much more rapidly.

Our final result will be that

$$\left| \frac{\pi}{4} - \sum_{i=0}^{n-1} \left(\frac{i!}{2} \prod_{j=0}^{i} \frac{1}{2j+1} \right) - \sum_{i=0}^{n} \left((-1)^i n! \prod_{j=0}^{n} \frac{1}{2(i+j)+1} \right) \right| \leq 4^{-n}$$

Since $10^{100} < 4^{167}$, we can take $n = 167$ to obtain an estimate for $\pi/4$ that is accurate to the 100 decimal places.

The key ingredient in this process is the *forward difference operator* Δ that maps a real sequence x to real sequence Δx, where $(\Delta x)_n = x_{n+1} - x_n$.

```
def Δ : (ℕ → ℝ) → (ℕ → ℝ) :=
                    fun x => (fun n => (x (n + 1) - x n))
```

Assume for now that we have a convergent alternating series that converges to a value L: $\sum_{n=0}^{\infty}(-1)^n y_n = L$. We prove first that $\sum_{n=0}^{\infty}(-1)^n(\Delta y)_n = y_0 - 2L$. Iterating this k times we obtain

$$\sum_{n=0}^{\infty}(-1)^n(\Delta^k y)_n = (-2)^k L + \sum_{j=0}^{k-1}(-2)^{k-(j+1)}(\Delta^j y)_0.$$

We also have, for any $k, n \in \mathbb{N}$, the following expression for the nth term of $\Delta^k y$

$$(\Delta^k y)_n = \sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i} y_{n+i}$$

In order to use these results for the Leibniz series, and in particular to give an explicit bound for the error term, we need to express this last alternating series as a product of positive terms. The following identity will help with this:

$$\sum_{j=0}^{k}(-1)^j\binom{k}{j}\frac{1}{2(i+j)+1} = 2^k k! \prod_{j=0}^{k}\frac{1}{2(i+j)+1}.$$

See the Lean file for more details.

Possible extension: find and formalise an example of a convergent alternating series for which Euler's forward difference operator does not change the rate of convergence.

3.2 Analysis 1 (`Analysis1`)

Mathlib is set up with limits defined primarily via filters.

To avoid this we have written some code to do much of 1st Year analysis in a more elementary style.

We have put this code in the folder `Projects/Analysis/Analysis1`.

If you are interested in doing a project in this area please talk to us and we will help you choose something suitable.

As examples there are two projects already setup: `Analysis/Accel` on accelerated convergence of series and `Analysis/IVT` on the intermediate value theorem.

We outline the main definitions and theorems from the `Analysis1` code below.

3.2.1 Sequences (`Sequences`)

The limit of a real sequence: we introduce the notation $\texttt{lim}_n$ `x a` to mean $x_n \to a$. Formally this is `sLim x a`.

```
def sLim (x : ℕ → ℝ) (a : ℝ) : Prop :=
  ∀ ε, 0 < ε → ∃ N, ∀ n, N ≤ n → |x n - a| < ε
```

Cauchy sequences:

```
def Cauchy (x : ℕ → ℝ) : Prop :=
  ∀ ε > 0, ∃ N, ∀ n ≥ N, |x n - x N| < ε
```

Bounded sequences:

```
def BdAbove (x : ℕ → ℝ) : Prop := ∃ b, ∀ n, x n ≤ b
def Bd (x : ℕ → ℝ) : Prop := BdAbove fun n => |x n|
```

The main theorems in this section are the following.

The *monotone convergence theorem*: if a real sequence is monotone increasing and bounded above then it converges.

```
theorem sLim_of_mono_bdAbove (hm : Monotone x) (hb : BdAbove x) :
     ∃ a, lim_n x a
```

The *Bolzano–Weierstrass theorem*: any bounded real sequence contains a convergent subsequence.

```
theorem bd_imp_conv_sub (hb : Bd x) :
    ∃ f : ℕ → ℕ, ∃ a, StrictMono f ∧ lim_n (x ∘ f) a
```

Any Cauchy sequence converges.

```
theorem sLim_of_cauchy (h : Cauchy x) : ∃ a, lim_n x a
```

We also prove numerous results about limits of sequences; both general and specific. For example:

```
lemma sLim_tail_sandwich (k : ℕ) (hx : lim_n x a) (hz : lim_n z a)
    (hb : ∀ n, k ≤ n → x n ≤ y n ∧ y n ≤ z n) : lim_n y a

theorem sLim_div (hx : lim_n x a) (hy : lim_n y b) (hnn : b ≠ 0) :
    lim_n (fun n => x n / y n) (a / b)

theorem one_over_nat : lim_n (fun n => (n + 1)^-1) 0
```

3.2.2 Supremum and Sets (Sets)

If a set $s \subseteq \mathbb{R}$ is bounded above and nonempty then there is a sequence of points in s that converges to its supremum.

```
theorem seq_sLim_sSup (hbd : BddAbove s) (hne : s.Nonempty) :
    ∃ x : ℕ → ℝ, (∀ n, x n ∈ s) ∧ lim_n x (sSup s)
```

3.2.3 Continuity (Continuity)

We define the limit of a real function at a to be l as follows (we also introduce the notation `lim_x f a l` for this).

```
def fLim (f : ℝ → ℝ) (a l : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x, x ≠ a → |x - a| < δ → |f x - l| < ε
```

We also define continuity at a point and on a set as follows.

```
def CtsAt (f : ℝ → ℝ) (a : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x, |x - a| < δ → |f x - f a| < ε

def CtsOn (f : ℝ → ℝ) (s : Set ℝ) : Prop :=
∀ a ∈ s,∀ ε > 0, ∃ δ > 0, ∀ x ∈ s, |x - a| < δ → |f x - f a| < ε
```

Our main theorems involving continuity are the following.

```
theorem extreme_value_max (hab : a ≤ b) (hf : CtsOn f (Icc a b)):
     ∃ c, c ∈ Icc a b ∧ ∀ x ∈ Icc a b, f x ≤ f c

theorem intermediate_value (hf : CtsOn f (Icc a b)) (hab : a < b)
     (hfab : f a < c ∧ c < f b) : ∃ x, x ∈ Ioo a b ∧ f x = c
```

3.2.4 Differentiation (Differentiation)

We define what it means for the derivative of a real function f at a point a to equal d. (We introduce the notation `∂ f a d` for this.)

```
def pderiv (f : ℝ → ℝ) (a d : ℝ) : Prop :=
  lim_x (fun h => (f (a + h) - f a) * h^-1) 0 d
```

A function is differentiable at a iff the following holds.

```
def dfble (f : ℝ → ℝ) (a : ℝ) : Prop := ∃ d, ∂ f a d
```

We also define the derivative on an open interval (a, b) as well as differentiability on (a, b) as follows.

```
def Ideriv (f d : ℝ → ℝ) (a b : ℝ) : Prop := ∀ x ∈ Ioo a b, ∂ f
     x (d x)

def Idfble (f : ℝ → ℝ) (a b : ℝ) : Prop := ∃ d : ℝ → ℝ,
    Ideriv f d a b
```

Our main theorems about differentiation are the following.

```
theorem rolle (hab : a < b) (hfc : CtsOn f (Icc a b))
(hfd : Idfble f a b) (hfeq : f a = f b) :
        ∃ c, c ∈ Ioo a b ∧ ∂ f c 0

theorem cauchy_mean_value (hab : a < b) (hfc : CtsOn f (Icc a b))
(hfd : Idfble f a b) (hgc : CtsOn g (Icc a b))
(hgd : Idfble g a b)  (hdnz : ∀ x ∈ Ioo a b, ¬∂ g x 0) :
    ∃ c, c ∈ Ioo a b ∧ ∃ d, ∂ f c d ∧
        ∃ e, ∂ g c e ∧ d * e⁻¹ = (f b - f a) * (g b - g a)⁻¹

theorem mean_value (hab : a < b) (hfc : CtsOn f (Icc a b))
(hfd : Idfble f a b) :
    ∃ c, c ∈ Ioo a b ∧ ∂ f c ((f b - f a) * (b - a)⁻¹)
```

In order to prove Taylor's theorem we introduce the following recursive definition of the kth derivative of a function f. We use the notation `Δ k f`.

```
def kdash : ℕ → (ℝ → ℝ) → ℝ → ℝ
| 0, f => f
| k + 1, f => fun a => if h : ∃ d, ∂ (kdash k f) a d
                        then h.choose else 0
```

So the 0th derivative of f is f; while the $(k+1)$th derivative of f is the derivative of the kth derivative if this exists and zero otherwise.

Thus a function f is k-times differentiable on the interval (a, b) iff the following holds.

```
def Ikdfble (k : ℕ) (f : ℝ → ℝ) (a b : ℝ) : Prop :=
∃ d : ℕ → ℝ → ℝ, ∀ j < k, ∀ x ∈ Ioo a b, ∂ (Δ j f) x (d j x)
```

We define the kth Taylor polynomial of f as follows.

```
def T (k : ℕ) (f : ℝ → ℝ) (x a : ℝ) : ℝ :=
   Σ i in range (k + 1), Δ i f a * (x - a) ^ i / i !
```

We prove a version of Taylor's theorem with a general remainder function, from which we can deduce the Cauchy and Lagrange forms of the remainder.

```
theorem taylor_cauchy (hax : a < x) (hfd : Ikdfble (k + 1) f a x)
 (hfc : ∀ j < k + 1, CtsOn (Δ j f) (Icc a x)) : ∃ z ∈ Ioo a x,
  f x - T k f x a = Δ (k + 1) f z * (x - z) ^ k * (x - a) / k !

theorem taylor_lagrange (hax : a < x)
 (hfd : Ikdfble (k + 1) f a x)
 (hfc : ∀ j < k + 1, CtsOn (Δ j f) (Icc a x)) : ∃ z ∈ Ioo a x,
 f x - T k f x a = Δ (k + 1) f z * (x - a) ^ (k + 1) / (k + 1)!
```

3.2.5 Series (Series)

A series converges iff its sequence of partial sums converges to a limit. We introduce the definition `Sums x` for this.

```
def Sums (x : ℕ → ℝ) : Prop :=
        ∃ a, lim_n (fun n => Σ i in range n, x i) a
```

We have most of the main theorems on convergence/divergence of series. For example:

```
lemma sums_nonneg_iff_bdAbove (hp : ∀ n, 0 ≤ x n) :
    Sums x ↔ BdAbove fun n => Σ i in range n, x i

theorem comparison_test (hp : ∀ n, 0 ≤ x n) (hxy : ∀ n, x n ≤ y
    n) (hs : Sums y) : Sums x

theorem sums_terms_to_zero (hx : Sums x) : lim_n (fun n => x n) 0

theorem sums_geo (ha : |a| < 1) : Sums fun n => a ^ n

theorem ratio_test (hnz : ∀ n, x n ≠ 0)
(hx : lim_n (fun n => |x (n + 1) / x n|) (|l|)) (hl : |l| < 1) :
    Sums (|x|)

theorem alternating_series_test (hanti : Antitone x)
(hlim: lim_n x 0) : Sums (fun n =>  (-1)^n * (x n))
```

In order to be able to define the exponential function we need to introduce the Cauchy product.

```
def cauchyProd (x y : ℕ → ℝ) : ℕ → ℝ :=
    fun n => Σ i in range (n + 1), x i * y (n - i)
```

We prove Merten's theorem.

```
theorem mertens (hx : lim_n (fun n => Σ i in range n, x i) a)
 (hy : lim_n (fun n => Σ i in range n, y i) b)
 (ha : Sums fun n => |x n|) :
    lim_n (fun n => Σ i in range n, cauchyProd x y i) (a * b)
```

We introduce convergence of a power-series with coefficients given by a real sequence $\mathtt{a} : \mathbb{N} \to \mathbb{R}$ at x.

```
def PowSums (a : ℕ → ℝ) (x : ℝ): Prop:= Sums fun n => a n * x^n
```

After proving the following lemma we can then define a function that returns the value of any power-series inside its radius of convergence.

```
lemma powSums_of_abs_lt (hc : PowSums a P) (hlt : |x| < |P|) :
    PowSums a x

def PowVal (a : ℕ → ℝ) (x : ℝ) : ℝ :=
  if h : ∃ P, PowSums a P ∧ |x| < |P| then
    (powSums_of_abs_lt h.choose_spec.1 h.choose_spec.2).choose
                                        else a 0
```

We introduce the notation Σ^p `a` to denote the function `PowVal a`.

In order to prove results about continuity and term-by-term differentiation of power-series we introduce uniform convergence of a sequence of functions on a set.

```
def UnifConv (f : ℕ → ℝ → ℝ) (g : ℝ → ℝ) (s : Set ℝ): Prop
    := ∀ ε > 0, ∃ N, ∀ x ∈ s, ∀ n, N ≤ n → |f n x - g x| < ε
```

Our main theorems are the following.

```
theorem ctsOn_of_unifConv_ctsOn (hu : UnifConv f g s)
(hc : ∀ n, CtsOn (f n) s) : CtsOn g s

theorem Ideriv_unifConv (hfu : UnifConv f g (Icc a b))
(hfud : UnifConv f' h (Icc a b))
(hfd : ∀ n, Ideriv (f (n + 1)) (f' n) a b) : Ideriv g h a b
```

In particular we can differentiate a power-series term-by-term inside its radius of convergence.

```
lemma powValDeriv (h : ∃ P, PowSums a P ∧ |x| < |P|) :
    ∂ (Σ^p a) x (Σ^p (fun n => (n + 1) * a (n + 1)) x)
```

We say that a power-series is *entire* if it converges everywhere.

```
def Entire (a : ℕ → ℝ) : Prop := ∀ x, PowSums a x
```

After establishing the next result we can formally define the exponential function and establish some of its basic properties.

```
theorem entire_exp (x : ℝ) : Sums fun n => x ^ n / n !
```

```
def exp (x : ℝ) : ℝ := (entire_exp x).choose
```

```
theorem exp_add (x y : ℝ) : exp (x + y) = exp x * exp y
```

Our final result on the exponential function is that it is its own derivative.

```
theorem expDeriv (x : ℝ) : ∂ exp x (exp x)
```

3.3 Cantor's intersection theorem (`CantorIntersection`)

A metric space (X, d) is said to be *complete* if every Cauchy sequence in (X, d) converges to a limit in X.

We introduce some basic definitions.

```
def TendsTo {X : Type} [MetricSpace X] (x : ℕ → X) (l : X) :
    Prop := ∀ ε > 0, ∃ N, ∀ n, N ≤ n → dist (x n) l < ε

def IsCauchy {X : Type} [MetricSpace X] (x : ℕ → X) : Prop :=
  ∀ ε > 0, ∃ N, ∀ m n, N ≤ m → m ≤ n → dist (x n) (x m) < ε

def Complete (X : Type) [MetricSpace X] : Prop :=
  ∀ (x : ℕ → X), IsCauchy x → ∃ l, TendsTo x l

def EventuallyConst {X : Type} (x : ℕ → X) : Prop :=
  ∃ N, ∀ n, N ≤ n → x n = x N
```

Our first result is a version of Cantor's intersection theorem.

Theorem. *If X is a complete metric space and $\{B(c_n, r_n)\}$ is decreasing sequence of non-empty closed balls whose radii tend to zero, then*

$$\bigcap_{n=0}^{\infty} B(c_n, r_n) \neq \emptyset.$$

The second part of the project is to formalise an example to prove that the condition 'radii tend to zero' is necessary in Cantor's theorem.

We define a distance function on $\mathbb{N}$ as follows.

```
def Ndist : ℕ → ℕ → ℝ :=
    fun m n => if m = n then 0 else 1 + 1 / (m + n)
```

We then prove that $(\mathbb{N}, \texttt{Ndist})$ is a metric space.

For $n \in \mathbb{N}$ we define the set $\texttt{Nball}(n) = \{n, n+1, ...\}$ as follows.

```
def Nball (n : ℕ) : Set ℕ := Set.Ici n
```

We then show that these sets form a strictly decreasing sequence of nonempty closed balls in $(\mathbb{N}, \texttt{Ndist})$.

We also prove that $(\mathbb{N}, \texttt{Ndist})$ is complete as follows.

Theorem. *Any Cauchy sequence in $(\mathbb{N}, \texttt{Ndist})$ is eventually constant and hence $(\mathbb{N}, \texttt{Ndist})$ is complete.*

Finally we show that the conclusion of Cantor's intersection theorem fails by proving that

$$\bigcap_{n=0}^{\infty} \texttt{Nball}(n) = \emptyset.$$

Possible extension: prove that we cannot replace *closed balls* by *open balls* in Cantors intersection theorem by giving an example of a complete metric space X and a decreasing sequence of non-empty open balls in X whose radii tend to zero yet have empty intersection.

3.4 Cantor Set (`CantorSet`)

A *ternary sequence* is $f : \mathbb{N} \to \{0, 1, 2\}$. We call a ternary sequence `Cantor` if it contains only 0's and 2's.

In Lean `Fin 3` is the type $\{0, 1, 2\}$. So a ternary sequence is $f : \mathbb{N} \to$ `Fin 3`.

We will want to have an ordering on our ternary sequences, and we use the *lexicographic* order (the natural dictionary order). In Lean this can be achieved by working in the type `Lex (ℕ → Fin 3)`. So our definition of a Cantor sequence is as follows.

```
def Cantor (f : Lex (ℕ → Fin 3)) : Prop :=
    ∀ n, f n = 0 ∨ f n = 2
```

We define a map from ternary sequences to $\mathbb{R}$ using the obvious base 3 expansion into $[0, 1]$.

```
def texpn (f : Lex (ℕ → Fin 3)) (n : ℕ) : ℝ :=
    (f n) * (3 : ℝ)⁻¹ ^ n.succ

def texp (f : Lex (ℕ → Fin 3)) : ℝ := Σ' k, texpn f k
```

Note that $\sum'$ is the `tsum` from Mathlib which is the limit of the infinite sum (if it exists), i.e.

$$\texttt{texp}(f) = \sum_{n=0}^{\infty} \frac{f(n)}{3^{n+1}}.$$

Note that `texp` is clearly not injective as a function from all ternary sequences. For example $010\overline{2}$ and $011\overline{0}$ both map to $4/27$. This example motivates the following two definitions.

```
def tmax (f : Lex (ℕ → Fin 3)) (n : ℕ) : Lex (ℕ → Fin 3) :=
    fun k => ite (k ≤ n) (f k) 2

def tmin (f : Lex (ℕ → Fin 3)) (n : ℕ) : Lex (ℕ → Fin 3) :=
    fun k => ite (k ≤ n) (f k) 0
```

Our first main theorem is the following.

Theorem. *The function* `texp` *is injective on the set of all Cantor sequences.*

This follows almost immediately from the following result.

Theorem. *If we order ternary sequences lexicographically and f, g are Cantor sequences satisfying $f < g$ then* $\texttt{texp}(f) < \texttt{texp}(g)$.

We then prove our second main result.

Theorem. *There is no surjection from the set of all Cantor sequences to $\mathbb{N}$.*

Putting these together and writing $\#A$ for the cardinality of a set A, we can prove that $\mathbb{R}$ is uncountable.

Theorem. *The reals are uncountable since:*

$$\#\mathbb{N} < \#\{f \mid f \ \ \mathit{Cantor}\} = \#\mathtt{texp}(\{f \mid f \ \ \mathit{Cantor}\}) \leq \#\mathbb{R}.$$

Possible extension: prove the following result characterising when distinct ternary sequences have expansions that sum to the same value.

```
theorem lt_of_texp_eq (hlt : f < g) (he : texp f = texp g) :
∃ N,∀ n < N, f n = g n ∧ (f N = 0 ∧ g N = 1 ∨ f N = 1 ∧ g N = 2)
                        ∧ tmax f N = f ∧ tmin g N = g
```

3.5 Contraction maps on $(\mathbb{N}, d_2)$ (`ContractionMap`)

A metric space (X, d) is *complete* iff every Cauchy sequence in X converges to a limit in X.

A map $f : X \to X$ is a *contraction* iff there exists $0 \leq K < 1$ such that $\forall x, y \in X,\ d(f(x), f(y)) \leq K d(x, y)$.

```
def ContractingWith (K : ℝ≥0) (f : α → α) :=
    K < 1 ∧ LipschitzWith K f
```

A *fixed point* of $f : X \to X$ is $x \in X$ such that $f(x) = x$.

```
def IsFixedPt (f : α → α) (x : α) := f x = x
```

Theorem (Banach)**.** *If (X, d) is a complete metric space and $f : X \to X$ is a contraction then f has a fixed point.*

In this project we will define a metric d_2 on $\mathbb{N}$ and use Banach's Fixed Point theorem to prove that $(\mathbb{N}, d_2)$ is not complete by showing that there exists a contraction map with no fixed point.

We start by giving a recursive definition of what we will call `log₂ n`.

```
def log₂ : ℕ → ℕ
  | 0 => 0
  | succ n =>
    if 2 ∣ succ n then log₂ (succ n / 2) + 1 else 0
```

Our first main goal is to prove that (for $n \neq 0$) we have

$$\log_2 n = \max\{k \in \mathbb{N} \mid 2^k \mid n\}.$$

We then define `ndist`$(m, n) = |m - n|$, where we want to consider m and n as integers rather than natural numbers.

```
def ndist : ℕ → ℕ → ℕ := fun m n => Int.natAbs ((m : ℤ) - n )
```

Next we define the 2-adic metric on $\mathbb{N}$ and prove that it is indeed a metric.

```
def d₂ : ℕ → ℕ → ℝ := fun m n =>
    if  (m = n)  then 0 else (1 / 2 ^ log₂ (ndist m n))
```

We then consider maps of the form $T : \mathbb{N} \to \mathbb{N}$, $T(n) = an + b$ and prove that if $a, b \neq 0$ then such a map has no fixed points. We also prove that for $a = 2$ such a map is a contraction. We then deduce our main result.

Theorem. *$(\mathbb{N}, d_2)$ is not complete.*

Possible extension: define continuity of $f : \mathbb{N} \to \mathbb{N}$ at $n \in \mathbb{N}$ in the obvious way.

```
def CtsAt (f : ℕ → ℕ) (n : ℕ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x, dist x n < δ → dist (f x) (f n) < ε
```

Prove that any polynomial with coefficients in $\mathbb{N}$ is continuous at every $n \in \mathbb{N}$.

```
theorem poly_cts (c : ℕ → ℕ) (m n : ℕ) :
        CtsAt (fun x => Σ j in range m, c j * x ^ j) n
```

You will probably want to prove a few basic results before doing this.

3.6 Dedekind cuts (`Dedekind`)

Dedekind cuts provide an explicit construction of the real numbers as a complete ordered field.

In this project we will define Dedekind cuts and establish some of their properties.

A *Dedekind cut* is a partition of the rationals $D = (A, A^c)$ satisfying four conditions:

- A is nonempty;
- A^c is nonempty
- A is a *down-set*, i.e. if $x \in A$ and $y < x$ then $y \in A$
- A has no maximum element.

Note that any Dedekind cut $D = (A, A^c)$ is uniquely determined by the set A, so we will often refer to a set $A \subset \mathbb{Q}$ as a Dedekind cut (where in fact we really mean (A, A^c)).

In Lean we introduce the following definition.

```
structure Dedekind where
  A          : Set ℚ
  nonempty   : A.Nonempty
  nonempty'  : Aᶜ.Nonempty
  down       : ∀ ⦃x y⦄, x < y → y ∈ A → x ∈ A
  no_max     : ∀ ⦃x⦄, x ∈ A → ∃ y ∈ A, x < y
```

Since we wish to *construct* the real numbers as Dedekind cuts, we should think of a Dedekind cut *as* a real number.

For any rational $q \in Q$ we can define the associated Dedekind cut

$$\texttt{rat}(q) = \{x \in \mathbb{Q} \mid x < q\}.$$

Examples of Dedekind cuts that don't correspond to a rational include

$$\{x \in \mathbb{Q} \mid x^2 < 2\},$$

and one of our examples is proving that this is indeed a cut.

There is a natural ordering on Dedekind cuts given by

$$D \leq E \iff D.A \subseteq E.A.$$

With this definition we can prove that we have a linear order and that the function `rat` : $\mathbb{Q} \to$ `Dedekind` is an order embedding of the rationals in the reals, i.e. `rat` is injective and satisfies $p \leq q \iff \texttt{rat}(p) \leq \texttt{rat}(q)$.

We can define addition by

$$D + E = \{x \in \mathbb{Q} \mid \exists d \in D.A, \exists e \in E.A, x = d + e\}$$

and zero to be

$$0 = \texttt{rat}(0) = \{x \in \mathbb{Q} \mid x < 0\}.$$

It is straightforward to check that addition is commutative and associative and 0 is the additive identity.

We can also show that addition respects the order: if $D \leq E$ then $C+D \leq C+E$.

Dedekind cuts are densely ordered: if $D < E$ then there exists F such that $D < F < E$.

So far all the properties we have established are shared by the rationals. The next one is not.

If S is a set of Dedekind cuts that is nonempty and bounded above then it has a supremum i.e. there exists a least upper bound for S.

The supremum of S is simply the union of the cuts in S:

$$\sup(S) = \bigcup_{D \in S} D.A.$$

Defining the infimum (without first defining subtraction/negation) is slightly more involved since the obvious candidate:

$$\bigcap_{D \in S} D.A,$$

may fail to define a cut. If $A_n = \{x \in \mathbb{Q} \mid x < 1/n\}$ and $S = \{(A_n, A_n^c) \mid n \in \mathbb{N}\}$ then

$$\bigcap_{n \in \mathbb{N}} A_n = \{x \in \mathbb{Q} \mid x \leq 0\},$$

which has a maximum element and hence is not a cut. We can get around this issue by defining

$$\inf(S) = \bigcap_{D \in S} D.A \setminus \mathsf{upperBounds}\left(\bigcap_{D \in S} D.A\right).$$

Using these we can prove Dedekind cuts form a conditionally complete linear order.

There are many possible extensions:

1. Prove that Dedekind cuts have the Archimedean property.
2. Define subtraction and negation and show that this gives a linear ordered additive commutative group

3.7 Set of continuity (`GdeltaCts`)

In Analysis 1 you saw the Thomae function $t : \mathbb{R} \to \mathbb{R}$, given by

$$t(x) = \begin{cases} 0, & x \notin \mathbb{Q}, \\ 1/q, & x = p/q \in \mathbb{Q},\ \gcd(p,q) = 1. \end{cases}$$

This is an example of a function that is continuous at x iff x is irrational.

A natural follow-up question is to ask what other sets can occur as the "set of continuity" of a function?

We will consider this question in the setting of metric spaces (X, d) and (Y, e).

```
def CtsAt (f : X → Y) (x : X) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ a, dist x a < δ → dist (f x) (f a) < ε

def setOfContinuity (f : X → Y) : Set X := {x | CtsAt f x}
```

So the Thomae function t satisfies `setOfContinuity`$(t) = \mathbb{R} \setminus \mathbb{Q}$.

If $x \in X$ and $\epsilon > 0$ then the *open ball* centre x and radius ϵ is the following subset of X:

```
ball x ε = {y | dist y x < ε}
```

A set $s \subseteq X$ is *open* iff

```
∀ (x : α), x ∈ s → ∃ ε, ε > 0 ∧ ball x ε ⊆ s
```

A set is said to be a G_δ-set iff it is the intersection of countably many open sets.

We will prove that for any function $f : X \to Y$, its `setOfContinuity`(f) is a G_δ-set.

Theorem. *If $f : X \to Y$ then there exists a sequence of open sets, $\{U_n\}_{n\in\mathbb{N}}$, such that*

$$\texttt{setOfContinuity}(f) = \bigcap U_n.$$

We can in fact write down an explicit sequence of open sets that will work for any given function.

```
def U (f : X → Y) (n : ℕ) : Set X :=
    {x | ∃ δ > 0, ∀ a b, dist x a < δ → dist x b < δ →
                              dist (f a) (f b) < ((n + 1) : ℝ)⁻¹}
```

We then consider the special case of $X = Y = \mathbb{R}$, and prove the following converse.

Theorem. *If $\{U_n\}_{n\in\mathbb{N}}$ is a sequence of open sets in $\mathbb{R}$ and $G = \bigcap U_n$ then there exists a function $\texttt{func}_U : \mathbb{R} \to \mathbb{R}$ such that*

$$\texttt{setOfContinuity}(\texttt{func}_U) = \bigcap U_n.$$

The function $\texttt{func}_U$ is defined as follows.

Let $G = \bigcap U_n$. For $x \in G$ we define $\texttt{func}_U(x) = 0$. For $x \notin G$ there is a smallest n such that $x \notin U_n$. The value of $\texttt{func}_U(x)$ is then defined to be $1/(n+1)$ or $-1/(n+1)$ depending on whether or not x is irrational. (In the definition below `(min_n h).choose` is this smallest n.)

```
def func (U : ℕ → Set ℝ) : ℝ → ℝ := fun x =>
  if h : x ∈ ⋂ n, U n then 0 else
    if (Irrational x) then ((min_n h).choose + 1)⁻¹
      else -((min_n h).choose + 1 : ℝ)⁻¹
```

Sketch of proof: if $x \in \bigcap U_n$ then $\texttt{func}_U(x) = 0$. Given $\epsilon > 0$, we can choose N such that $1/(N+1) < \epsilon$. Since each U_n is open there exist $\delta_n > 0$ such that $\texttt{ball x } \delta_\texttt{n} \subseteq \texttt{U n}$. Choosing $\delta \texttt{ = min } \{\delta_i \texttt{ | i } \leq \texttt{N}\} > 0$ we can show that $|f(y)| \leq 1/(N+1) < \epsilon$ for any y satisfying $\texttt{dist x y < } \delta$ and hence $\texttt{func}_U$ is continuous at x.

Conversely we need to show that if $\texttt{func}_U$ is continuous at x then $x \in \bigcap U_n$. Suppose for a contradiction that $x \notin \bigcap U_n$ and let N be minimal such that $x \notin U_N$. By definition $|\texttt{func}_U(x)| = 1/(N+1)$. By choosing an appropriate value of ϵ and considering the cases of whether or not $x \in \mathbb{Q}$ we can derive a contradiction.

See the Lean file for some useful lemmas.

Possible extension: give an example of a metric space (X, d) and a G_δ-set $G \subseteq X$ such that for any metric space (Y, e) and for any function $f : X \to Y$ we have

$$G \neq \texttt{setOfContinuity}(f).$$

3.8 Fractal convergence (IFS)

The Cantor set is often described as the "limit" of the following process: start with the interval $[0, 1]$; delete the middle third; now repeat with each remaining interval.

In this project we will prove that this limit is well defined and does not depend on the intial set (so long as that set is nonempty and compact).

Recall that a metric space X is said to be *complete* iff every Cauchy sequence in X converges to a limit in X. A mapping $f : X \to X$ is said to be *K-Lipschitz*, where $K \geq 0$, iff $\forall x, y \in X, d(f(x), f(y)) \leq Kd(x, y)$. If $0 \leq K < 1$ then f is a *contraction* mapping.

Theorem (Banach fixed point)**.** *Any contraction mapping F on a nonempty complete metric space X has a unique fixed point, i.e. there exists a unique solution $x^* \in X$ to $F(x^*) = x^*$.*

The fixed point x^* can be found as follows: let $x_0 \in X$ and define $x_{n+1} = F(x_n)$ then $\lim_{n\to\infty} x_n = x^*$.

In order to apply this result to sequences of sets we need a metric on sets: the Hausdorff metric.

In Lean metric spaces are generalized in two simple ways: a `PseudoMetricSpace` is a metric space in which $d(x, y) = 0$ may hold for some $x \neq y$; while a `PseudoEMetricSpace` is a `PseudoMetricSpace` in which $d(x, y) = +\infty$ may hold.

Lean defines the minimal distance between a point and a set as follows ($\sqcap$ denotes infimum).

```
def infEdist (x : α) (s : Set α) : ℝ≥0∞ :=
  ⨅ y ∈ s, edist x y
```

The Hausdorff distance between two sets is then defined as follows ($\sqcup$ denotes supremum).

```
def hausdorffEdist (s t : Set α) : ℝ≥0∞ :=
  (⨆ x ∈ s, infEdist x t) ⨆ (⨆ y ∈ t, infEdist y s)
```

One way of thinking about Hausdorff distance is in terms of r-neighbourhoods of sets. Given a set s in a metric space X and $r \geq 0$ we can define this as

$$N_r(s) = \{y \in X | \exists x \in s, d(x, y) \leq r\}.$$

With this definition the Hausdorff distance satisfies

$$d_H(s, t) < r \text{ iff } s \subseteq N_r(t) \text{ and } t \subseteq N_r(s).$$

One crucial property we will require is that if X is a complete metric space then the Hausdorff distance defines a complete metric on the Nonempty compact subsets of X.

We can now introduce the definition of a *contractive iterated function system* (or IFS for short). This consists of a finite nonempty set of contraction mappings from X to itself.

```
structure IFS (X : Type) (K : ℝ≥0)  [MetricSpace X] : Type where
A : Finset (X → X)
nonempty : A.Nonempty
lip : ∀ ⦃f⦄, f ∈ A → LipschitzWith K f
con : K < 1
```

For example, the Cantor IFS on $X = \mathbb{R}$ could be defined as $A = \{f_0, f_1\}$, with $f_0(x) = x/3$ and $f_1(x) = 2/3 + x/3$ and $K = 2^{-1}$.

Given such an IFS $\mathcal{F}$ and a nonempty compact set $s \subseteq X$ we can define a map taking s to the finite union of its images under the functions of the IFS

$$\mathcal{F}(s) = \bigcup_{f \in \mathcal{F}.A} f(s).$$

To make this definition in Lean we need to prove that this does indeed define a nonempty compact set in X.

Given any continuous function f and a nonempty compact set s the image $f(s)$ is also compact and non-empty so we introduce the following definition.

```
def Continuous.map {f : X → X} (h : Continuous f)
(s : NonemptyCompacts X) : NonemptyCompacts X where
    carrier    := f '' s
    isCompact'  := by sorry
    nonempty'   := by sorry
```

We can then define the map of the IFS as described above.

```
def map (𝓕 : IFS X K) (s : NonemptyCompacts X) :
 NonemptyCompacts X where
    carrier   := ⋃ f ∈ 𝓕.A, f '' s
    isCompact' := by sorry
    nonempty'  := by sorry
```

Having shown that any IFS induces a contraction mapping on the space of NonemptyCompacts X we can define the limit of an IFS using the completeness of the Hausdorff metric and the Banach fixed point theorem.

```
def limit (𝓕 : IFS X K) : NonemptyCompacts X := by sorry
```

Our final theorem gives a bound on the rate of convergence to this limit of iterating $\mathcal{F}$ starting from an arbitrary nonempty compact set s.

Theorem. *If $\mathcal{F}$ is a contractive IFS with Lipschitz factor K on a complete metric space X, and s is a nonempty compact subset of X then we have the*

following bound in Hausdorff distance of the convergence of $\mathcal{F}^{[n]}(s)$ to the limit of $\mathcal{F}$:

$$d_H(\mathcal{F}^{[n]}(s), \mathcal{F}.\texttt{limit}) \leq \frac{K^n d_H(s, \mathcal{F}(s))}{1-K}.$$

We give some famous examples including the Cantor set and Siepinski's triangle.

```
def Cantor : IFS ℝ 3⁻¹ where
A := {fun x => 3⁻¹ * x, fun x => 3⁻¹ * (x + 2)}

def Sierpinski : IFS (ℝ × ℝ) 2⁻¹ where
A := {fun x => 2⁻¹ * x, fun x => 2⁻¹ * x + ⟨2⁻¹,0⟩,
            fun x => 2⁻¹ * x + 4⁻¹*⟨1, Real.sqrt 3⟩}
```

Possible extension: prove that if you iterate the Cantor IFS, starting with the singleton set $\{0\}$, then after n steps you are within Hausdorff distance at most $1/3^n$ of the Cantor set (the limit of the Cantor IFS).

3.9 The intermediate value theorem (IVT)

In this project we formalise the standard proof of the Intermediate Value Theorem.

Theorem. *If $a < b$ and $f : \mathbb{R} \to \mathbb{R}$ is continuous on $[a, b]$, and $f(a) < c < f(b)$ then there exists $d \in (a, b)$ such that $f(d) = c$.*

We then apply this to prove the following simple fixed point result.

Theorem. *If $a < b$ and $f : \mathbb{R} \to \mathbb{R}$ is continuous on $[a, b]$, and $f([a, b]) = [a, b]$ then there exists $x \in [a, b]$ such that $f(x) = x$.*

We use the standard definition of continuity of a function $f : \mathbb{R} \to \mathbb{R}$ on a set s.

```
def CtsOn (f : ℝ → ℝ) (s : Set ℝ) : Prop :=
∀ x ∈ s,∀ ε > 0, ∃ δ > 0, ∀ z ∈ s, |z - x| < δ → |f z - f x| < ε
```

We prove first that this is equivalent to the following definition (this makes the proof of the IVT slightly easier).

```
def CtsOn' (f : ℝ → ℝ) (s : Set ℝ) : Prop :=
∀ x ∈ s,∀ ε > 0, ∃ δ > 0,∀ z ∈ s, |z - x| ≤ δ → |f z - f x| < ε
```

Our statement of the intermediate value theorem in Lean is as follows.

```
theorem intermediate_value (hf : CtsOn f (Icc a b)) (hab : a < b)
    (hfab : f a < c ∧ c < f b) : ∃ x, x ∈ Ioo a b ∧ f x = c
```

Note that in Lean the closed interval $[a, b]$ is called `Icc a b` while the open interval (a, b) is `Ioo a b`.

For the proof of the IVT you will need to use results about the `sSup` function in Lean. This is the supremum of a set (of real numbers in our case). In Lean this is defined for all subsets of $\mathbb{R}$, not just those that are non-empty and bounded above.

```
theorem sSup_def (S : Set ℝ) :
sSup S = if h : S.Nonempty ∧ BddAbove S then Classical.choose
                    (exists_isLUB S h.1 h.2) else 0
```

You shouldn't need to use this definition directly, instead use results from Mathlib such as the following.

```
theorem le_csSup (h₁ : BddAbove s) (h₂ : a ∈ s): a ≤ sSup s

theorem csSup_le (h₁ : s.Nonempty) (h₂ : ∀ b ∈ s, b ≤ a) :
    sSup s ≤ a
```

The proof of the IVT starts as follows.

Let $S = \{x \mid x \in [a, b] \text{ and } f(x) < c\}$. Prove that S is nonempty and bounded above. We can now talk about the supremum of S and it agrees with `sSup S`

in Lean. We need to prove that `sSup S ∈ Ioo a b` and that `f(sSup S) = c`. The proof of `f(sSup S) = c` can be done by cases, obtaining a contradiction if `f(sSup S) < c` or `f(sSup S) > c`.

You should write out the full proof in as much detail as possible before starting to formalise it.

Possible extension. Let $f : \mathbb{R} \to \mathbb{R}$ be continuous on $[-1, 1]$ and satisfy $f(-1) = f(1) = 0$. Prove that if $a, b \in \mathbb{N}$ then there exists $c \in (-1, 1)$ such that

$$f(c) = \frac{c^{2a+1}}{1 - c^{b+1}}.$$

3.10 Measure Theory (`MeasureTheory`)

3.10.1 Vitali Set (`VitaliSet`)

In this project (which doesn't require any actual measure theory) we will formalise Vitali's proof that, with any sensible definition of a measure, not all subsets of $\mathbb{R}$ can be measurable.

We say that a function μ : `Set` $\mathbb{R} \to [0, +\infty]$ *is a Vitali-measure* if it has the following properties.

```
structure IsVitaliMeasure (μ : Set ℝ → ℝ≥0∞) : Prop where
mono : ∀ s t, s ⊆ t → μ s ≤ μ t
unit_length : μ (Icc 0 1) = 1
translate_invariant : ∀ s : Set ℝ, ∀ x : ℝ, μ (s +ᵥ x) = μ s
ctble_add : ∀ (F : ℕ → Set ℝ), (∀ {i j}, i ≠ j →
    Disjoint (F i)  (F j)) → μ (⋃ n, F n) = Σ' n, μ (F n)
```

These conditions say, respectively, that μ is monotone; $\mu([0,1]) = 1$; μ is translation invariant and μ is countably additive i.e. given a countable sequence of pairwise disjoint sets the measure of their union is the sum of their measures.

The main goal of this project is to prove that no Vitali-measure exists.

We first introduce a relation on $\mathbb{R}$ by `eqv` $x\ y \iff x - y \in \mathbb{Q}$.

```
def eqv : ℝ → ℝ → Prop :=
    fun x y => (x - y) ∈ range ((↑) : ℚ → ℝ)
```

We prove that this is an equivalence relation which we call `Vitali`. In Lean we register this equivalence relation using a `Setoid` instance.

```
instance Vitali : Setoid ℝ
```

We prove that each equivalence class contains an element in $[0, 1/2]$. We can then define a function that, given an equivalence class, chooses an element in $[0, 1/2]$.

```
def half_rep : Vitali.classes → ℝ :=
by
  classical
  intro c
  exact (exists_half_rep c.prop).choose
```

We then define a set of reals $\mathcal{V}$ by choosing a single element from each equivalence class in $[0, 1/2]$ using this function.

```
def 𝒱 : Set ℝ := range half_rep
```

We then establish that

(A) $\mathcal{V} \subseteq [0, 1/2]$

(B) $\bigcup_{q\in\mathbb{Q}}(\mathcal{V}+q)=\mathbb{R}$.

(C) If $p,q\in\mathbb{Q}$ and $p\neq q$ then $\mathcal{V}+p$ and $\mathcal{V}+q$ are disjoint.

Note that (A) implies that for all $n\in\mathbb{N}$, $\mathcal{V}+\frac{1}{n+2}\subseteq[0,1]$. Hence we also have

(D) $\bigcup_{n=0}^{\infty}\mathcal{V}+\frac{1}{n+2}\subseteq[0,1]$

Our main theorem is the following.

Theorem. *No Vitali-measure exists.*

The proof is as follows. Suppose that μ is a Vitali-measure. We have, using translation invariance; countable additivity with (C); and monotonicity with (D) that

$$\sum_{n=0}^{\infty}\mu(\mathcal{V})=\sum_{n=0}^{\infty}\mu\left(\mathcal{V}+\frac{1}{n+2}\right)=\mu\left(\bigcup_{n=0}^{\infty}\mathcal{V}+\frac{1}{n+2}\right)\leq\mu([0,1])=1.$$

But the LHS is an infinite sum of constant nonnegative terms, hence $\mu(\mathcal{V})=0$. Using the fact that $\mathbb{Q}$ is countable (B) and (C), together with translation invariance and countable additivity we have

$$\mu(\mathbb{R})=\mu\left(\bigcup_{q\in\mathbb{Q}}\mathcal{V}+q\right)=\sum_{q\in\mathbb{Q}}\mu(\mathcal{V})=\sum_{q\in\mathbb{Q}}0=0.$$

Which is impossible, since $\mu([0,1])\subseteq\mathbb{R}$ and $\mu([0,1])=1$.

Possible extension: explore what happens if we weaken the condition of *countably additive* to *finitely additive*.

3.11 Topology (Topology)

3.11.1 Alexandrov topology and orders (Alexandrov)

This project is on general topology. We explore some basic definitions such as discrete and Alexandrov topologies with a focus on the kinds of topologies that arise in computer science. In particular we show how to construct a preorder from a topology and how to construct a topology from a preorder; characterising when these constructions are inverses of each other.

A *topology* τ on a set α is a family of subsets of α that contains $\emptyset$, `univ` and is closed under pairwise intersections and arbitrary unions.

The sets in this family are called the *open* sets. (A set is *closed* iff its complement is open.)

The definition of a topological space in Lean looks something like this.

```
class TopologicalSpace (α : Type) where
  IsOpen : Set α → Prop
  isOpen_univ : IsOpen univ
  isOpen_inter : ∀ s t, IsOpen s → IsOpen t → IsOpen (s ∩ t)
  isOpen_sUnion : ∀ s, (∀ t ∈ s, IsOpen t) → IsOpen (⋃₀ s)
```

A *discrete* topological space is one in which every set is open.

```
def IsDiscrete (α : Type) [TopologicalSpace α] : Prop :=
  ∀ s : Set α, IsOpen s
```

In any topological space every *union* of open sets is open. A space in which every *intersection* of open sets is open is said to be *Alexandrov*.

```
def IsAlexandrov (α : Type) [TopologicalSpace α] :=
  ∀ S : Set (Set α), (∀ s ∈ S, IsOpen s) → IsOpen (⋂₀ S)
```

In an Alexandrov space the intersection of all the open sets containing a point `a` is itself open (and hence is the unique smallest open set containing `a`). We call this `Inbhd a`.

```
def Inbhd (a : α) : Set α := ⋂₀ { u : Set α | a ∈ u ∧ IsOpen u }
```

In fact we have the following characterisation of Alexandrov spaces.

Theorem. *A topological space α is Alexandrov iff $\forall a \in \alpha$, `Inbhd a` is open.*

Separation axioms are a way imposing extra conditions on a topological space that tell us how easy it is to distinguish between points (or sets) using the topology.

They start with T_0-spaces and go up to T_5-spaces. We will only really consider T_0 and T_1-spaces.

A T_0-space is a topological space in which $x \neq y$ implies there exists an open set u such that either $x \in u$ and $y \notin u$ or $x \notin u$ and $y \in u$.

A T_1-space is a topological space in which $x \neq y$ implies there exists an open set u such that $x \in u$ and $y \notin u$.

In Lean this is defined equivalently as every singleton set is closed, which in turn is equivalent to saying that the complement of every singleton set is open `∀ (a : α), IsOpen {a}`^{c}.

Theorem. *Any Alexandrov T_1-space is discrete.*

One important place in which Alexandrov spaces arise is when constructing a topological space from a preorder.

Given a topological space α we can define a "$\leq$" on on α by

```
a ≤ b ↔ Inbhd b ⊆ Inbhd a
```

We show this gives a Preorder (which we will call `toPre`). Moreover, if we start with a T_0-space, then `toPre` is in fact a PartialOrder.

We then consider how to define a topology on a Preorder. To define a topology we need to describe the open sets. We define them as follows.

```
IsOpen s ↔ { s | ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, x ≤ y → y ∈ s }
```

We prove that this does indeed give a topological space, which we call `toTop`.

We show `toTop` is in fact Alexandrov. Moreover if the Preorder we start with is a Partial order then `toTop` is a T_0-space.

We then consider when these constructions are inverses of each other.

We prove first that if we start with a Preorder β and then form `toPre (toTop` β) then we recover the original Preorder on β.

```
theorem toPre_of_toTop_eq_Pre [Preorder β] (a b : β) :
    a ≤ b ↔ (Inbhd b ⊆ Inbhd a)
```

Since `toTop` is Alexandrov we cannot hope to prove an analogous result starting from a general topological space. However we have the next best thing. If we start with an Alexandrov space β and form the Preorder `toPre` β and then form the topological space from this Preorder, i.e. `toTop (toPre` β`)` we recover the original topological space.

```
theorem toTop_of_toPre_eq_Top_of_Alexandrov [TopologicalSpace β]
  (hA : IsAlexandrov β) (s : Set β):
  IsOpen s ↔ ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, x ≤ y → y ∈ s
```

Possible extension: consider the two element type `Point`:

```
inductive Point where
| bot : Point
| top : Point
```

Construct a topological space on `Point` by giving it an order with `bot ≤ bot`, `bot ≤ top` and `top ≤ top` and have Lean infer the topology from this using

`toTop`. Prove that given any topological space α there is a equivalence between the open sets in the topology of α and the continuous functions `f : α → Point`.

```
{s : Set α // IsOpen s} ≃ {f : α → Point // Continuous f}
```

For this you need to use the definition of a continuous function between topological spaces. If $f : \alpha \to \beta$ with α and β topological spaces, then we say that f is *continuous* iff the preimage of every open set in β is open in α.

```
Continuous f ↔ ∀ (s : Set β), IsOpen s → IsOpen (f ⁻¹' s)
```

3.12 Uniform convergence boundedness and continuity (`Uniform`)

Let X and Y be metric spaces and suppose that $\{f_n\}$ is a sequence of functions from X to Y. In this project we formalise some results concerning uniform convergence.

Theorem. *If $f_n \to g$ uniformly on a set $s \subseteq X$, and each f_n is bounded on s, then g is bounded on s.*

Theorem. *If $f_n \to g$ uniformly on a set s, and each f_n is continuous on s, then g is continuous on s.*

We introduce the following definitions in Lean. The first is of pointwise convergence on a set; while the second is of uniform convergence on a set. The only difference between these two definitions is the order of the quantifiers.

```
def PtwiseConv (f : ℕ → X → Y) (g : X → Y) (s : Set X) : Prop
:= ∀ x ∈ s, ∀ ε > 0, ∃ N, ∀ n, N ≤ n → dist (f n x) (g x) < ε

def UnifConv (f : ℕ → X → Y) (g : X → Y) (s : Set X) : Prop :=
  ∀ ε > 0, ∃ N, ∀ x ∈ s, ∀ n, N ≤ n → dist (f n x) (g x) < ε

def BddOn (f : X → Y) (s : Set X) : Prop :=
  ∃ B, ∀ a b, a ∈ s → b ∈ s → dist (f a) (f b) ≤ B

def CtsOn (f : X → Y) (s : Set X) : Prop := ∀ a ∈ s, ∀ ε > 0,
    ∃ δ > 0, ∀ b ∈ s, dist b a < δ → dist (f b) (f a) < ε
```

As an application let $X = Y = \mathbb{R}$. Prove that if $F_n = n/(nx+1)$ then F_n is bounded on $(0,2)$ and $F_n \to 1/x$ pointwise on $(0,2)$, but F_n does not converge uniformly to $1/x$ on $(0,2)$ since the pointwise limit is unbounded.

Possible extension: give an example of a sequence of functions which are continuous on $[0,2]$ and converge pointwise to a continuous function on $[0,2]$ but for which the convergence is not uniform.

A potential example is the following sequence of functions:

$$f_n(x) = \begin{cases} (n+1)^2 x, & \text{if } x \leq (n+1)^{-1}, \\ 2(n+1) - (n+1)^2 x, & \text{if } (n+1)^{-1} < x \leq 2(n+1)^{-1}, \\ 0, & \text{otherwise.} \end{cases}$$

4 Combinatorics (`Combinatorics`)

4.1 Intersecting Families of sets (`Intersecting`)

We say that a family of sets $\mathcal{A}$ is *intersecting* if $A, B \in \mathcal{A}$ implies $A \cap B \neq \emptyset$.

Given a finite nonempty type α let $\mathcal{P}(\alpha) = \{A \mid A \subseteq \alpha\}$ denote its powerset. We say that an intersecting family $\mathcal{A} \subseteq \mathcal{P}(\alpha)$, is a *maximal intersecting family* iff $\mathcal{A}$ is intersecting and $\mathcal{A}$ is not properly contained in any larger intersecting family from $\mathcal{P}(\alpha)$.

The main aim of this project is to prove the following theorem.

Theorem. *If $\mathcal{A} \subseteq \mathcal{P}(\alpha)$ is a maximal intersecting family then $|\mathcal{A}| = 2^{||\alpha||-1}$.*

We define an intersecting family as follows.

```
def Intersecting (𝒜 : Finset (Finset α)) : Prop :=
  ∀ ⦃A⦄, A ∈ 𝒜 → ∀ ⦃B⦄, B ∈ ℬ → ¬Disjoint A B
```

We then define a maximal intersecting family as follows.

```
def MaximalIntersecting (𝒜 : Finset (Finset α)) : Prop :=
 Intersecting 𝒜 ∧ ∀ ⦃ℬ⦄, 𝒜 ⊆ ℬ → Intersecting ℬ → 𝒜 = ℬ
```

If `A : Finset α` and α is finite (i.e. we have a `[Fintype α]` instance) then the complement of `A` (in α) is also a `Finset α` and we can write $\mathtt{A}^c$ for it in Lean.

Given a family $\mathcal{A}$ we need some notation for the family of the complements of members of $\mathcal{A}$. This is not, in general, the same as the complement of $\mathcal{A}$ in $\mathcal{P}(\alpha)$ but we will abuse notation by defining this to be

$$\mathcal{A}^{\mathtt{c}} := \{\mathtt{A}^{\mathtt{c}} \mid \mathtt{A} \in \mathcal{A}\}.$$

In Lean this is achieved by overwriting the standard definition of complement in this particular case.

```
instance : HasCompl (Finset (Finset α)) :=
⟨fun 𝒜 => 𝒜.image fun A => Aᶜ⟩
```

Beware that the complement operator ^{c} is now being used in two different ways (depending on whether it is applied to `𝒜 : Finset (Finset α)` or to `A : Finset α`).

Our first theorem is the following upper bound.

```
theorem card_le_of_int (hI : Intersecting 𝒜) : |𝒜| ≤  2^||α||-1
```

The idea behind the proof of this theorem is easy.

Given any member A of an intersecting family $\mathcal{A}$ we know that $A^c \notin \mathcal{A}$ (since A and A^c are disjoint). This implies (with our abuse of notation) that $\mathcal{A}$ and $\mathcal{A}^c$ are disjoint as families of sets in $\mathcal{P}(\alpha)$. Moreover $|\mathcal{A}| = |\mathcal{A}^c|$ since the map $A \mapsto A^c$ is bijective. This result then follows from the fact that $|\mathcal{P}(\alpha)| = 2^{||\alpha||}$.

To deduce our main theorem we need to prove that if $\mathcal{A}$ is a maximal intersecting family then in fact $\mathcal{P}(\alpha) = \mathcal{A} \cup \mathcal{A}^c$.

After establishing the main theorem we then show that there exist maximal intersecting families for any finite nonempty α by giving the following instance.

```
instance inhabited_max_int :
  Inhabited { 𝒜 : Finset (Finset α) // MaximalIntersecting 𝒜 }
```

Possible extension: prove the following special case of the Erdős–Ko-Rado theorem. If $\mathcal{A}$ is an intersecting family of k-sets from a set of size $2k$ and $k \geq 1$ then $|\mathcal{A}| \leq \binom{2k-1}{k-1}$.

4.2 Ramsey Theory (`Ramsey`)

4.2.1 Ramsey's theorem for two colours (`Graph2Col`)

The aim of this project is to prove Ramsey's theorem for graphs: for any $s, t \in \mathbb{N}$, there exists $N \in \mathbb{N}$ such that any red/blue-edge-colouring of the complete graph K_N contains a red K_s or a blue K_t. Denoting the smallest such N by $R(s,t)$ we will prove the Erdős–Szekeres bound

$$R(s+1, t+1) \leq \binom{s+t}{t}.$$

Although this result is usually stated in terms of complete graphs it is more easily described as a result about 2-colourings of pairs of natural numbers, and this formulation will allow us to later generalise the problem to hypergraphs. For any set X and $r \in \mathbb{N}$ let $X^{(r)}$ denote the family of all subsets of X of size r

$$X^{(r)} = \{A \subseteq X \mid |A| = r\}.$$

We use `Fin 2 = {0,1}` as our colours (we will think of 0 as red and 1 as blue). A 2-colouring will be any map `col : Finset ℕ → Fin 2`. (Notice that our 2-colourings assign a colour to every finite subset of $\mathbb{N}$, this is simply to make things easier in the formalisation.)

If `A : Finset ℕ` then the Lean for $\mathtt{A}^{(2)}$ is `A.powersetLen 2`.

Thus a 2-colouring `col` is monochromatic with colour `c` on $\mathtt{A}^{(2)}$ iff the following holds. (We will say simply $\mathtt{A}^{(2)}$ is red/blue.)

```
def monoOn (A : Finset ℕ) (col : Finset ℕ → Fin 2) (c : Fin 2) :
 Prop:= ∀ (e : Finset ℕ), e ∈ A.powersetLen 2  → col e = c
```

We then define what it means for a 2-colouring `col` to be a `RamseyCol` for $N, s, t \in \mathbb{N}$.

```
def RamseyCol (col : Finset ℕ → Fin 2) (N s t : ℕ) : Prop :=
∀ {V : Finset ℕ}, N ≤ |V| →
(∃ (A : Finset ℕ), A ⊆ V ∧ ((A.monoOn col 0 ∧ s ≤ |A|)
                                  ∨ (A.monoOn col 1 ∧ t ≤ |A|)))
```

This says that, under the 2-colouring `col`, *every* finite subset of $\mathbb{N}$ of size at least N contains a set `A` such that $\mathtt{A}^{(2)}$ is red and $|\mathtt{A}| \geq \mathtt{s}$ or $\mathtt{A}^{(2)}$ is blue and $|\mathtt{A}| \geq \mathtt{t}$.

Finally we define what it means for $N \in \mathbb{N}$ to be `RamseyOf` for $s, t \in \mathbb{N}$.

```
 def RamseyOf (N s t : ℕ): Prop:=
    ∀ (col : Finset ℕ → Fin 2), RamseyCol col N s t
```

This says that in *every* 2-colouring, every subset of size at least N contains a set `A` such that $\mathtt{A}^{(2)}$ is red and $|\mathtt{A}| \geq \mathtt{s}$ or $\mathtt{A}^{(2)}$ is blue and $|\mathtt{A}| \geq \mathtt{t}$.

[Note that this definition appears superficially stronger than the usual definition but is easily seen to be equivalent, moreover it is much easier to work with in Lean.]

The main content of Ramsey's theorem is that for any $s, t \in \mathbb{N}$ there exists $N \in \mathbb{N}$ such that `RamseyOf N s t` holds, and hence we can define the Ramsey number `R(s,t) = min { N | RamseyOf N s t}`.

The proof of Ramsey's theorem uses an inductive argument. Given a 2-colouring `col`, a finite set `V : Finset ℕ` and `n ∉ V` we will want to consider the coloured neighbourhoods of `n` in `V`, i.e. the sets of vertices `w ∈ V` such that `{w,n}` is red/blue.

```
def nbhd  (col : Finset ℕ → Fin 2) (c : Fin 2) (V : Finset ℕ)
(n : ℕ) : Finset ℕ := V.filter (fun w => col (insert w {n}) = c)
```

Notice that in our definition we do not insist that `n ∉ V`, however this will be the case in our applications.

The other definitions we require relate to the obvious symmetry between colours red 0 and blue 1.

```
def notC (c : Fin 2) : Fin 2 := if (c = 0) then 1 else 0

def notCol (col : Finset ℕ → Fin 2) :
                    Finset ℕ → Fin 2 :=  notC ∘ col
```

The key lemma is the following step.

```
lemma Ramsey_step (hab : 1 ≤ a + b): RamseyOf a s (t + 1) →
    RamseyOf b (s + 1) t → RamseyOf (a + b) (s + 1) (t + 1)
```

We sketch the proof. Suppose that `a` and `b` satisfy `RamseyOf a s (t + 1)` and `RamseyOf b (s + 1) t`. We want to prove that `RamseyOf (a + b) (s + 1) (t + 1)` holds. Take a 2-colouring `col` and a set `W` $\subseteq \mathbb{N}$ of size at least `a + b` and remove an element `n` from `W` to obtain `V`. Now consider the red/blue neighbourhoods of `n` inside `V`. Either the red neighbourhood of `n` in `V` has size at least `a` or the blue neighbourhood has size at least `b`. In the former case, since `RamseyOf a s (t + 1)` we can find a set `C` $\subseteq$ `V` such that either `s` $\leq |$`C`$|$ and `C`$^{(2)}$ is red or `t` $+ 1 \leq |$`C`$|$ and `C`$^{(2)}$ is blue. In the latter subcase we are done; in the former subcase we can check that `s` $+ 1 \leq |$`C` $\cup \{$`n`$\}|$ and (`C` $\cup \{$`n`$\}$)$^{(2)}$ is red. The other case follows similarly using the assumption that `RamseyOf b (s + 1) t`.

The Erdős–Szekeres version of Ramsey's theorem is then proved by a double induction.

```
theorem Erdos_Szekeres_Ramsey (s t : ℕ)  :
    RamseyOf ((s + t).choose s) (s + 1) (t + 1)
```

Possible extension: prove some constructive lower bounds for small values of s and t.

4.2.2 Ramsey's theorem for k colours (`KColourRamsey`)

We will assume Ramsey's theorem for two colours in the form of the Erdős–Szekeres bound (see project `Graph2Col`) and establish Ramsey's theorem for k colours.

Although this theorem is usually stated in terms of complete graphs it is more easily described as a result about k-colourings of pairs of natural numbers, and this formulation will allow us to later generalise the problem to hypergraphs. For any set X and $r \in \mathbb{N}$ let $X^{(r)}$ denote the family of all subsets of X of size r

$$X^{(r)} = \{A \subseteq X \mid |A| = r\}.$$

Our set of `k` colours will be `Fin k` (the type of natural numbers less than `k`). We will refer to `col : Finset ℕ → Fin k` as a `k`-colouring. This assigns a colour to every finite subset of $\mathbb{N}$ although we will only care about the colours of pairs $\mathbb{N}^{(2)}$. Given a finite set `A` $\subseteq \mathbb{N}$; a k-colouring `col` and a colour `c` we have the basic definition of a monochromatic clique, i.e. a set for which all pairs receive the same colour.

```
def KmonoOn (A : Finset ℕ) (col : Finset ℕ → Fin k)
(c : Fin k) : Prop :=  ∀ e, e ∈ A.powersetLen 2 → col e = c
```

In the 2-colour version of Ramsey's theorem we had parameters `s`, `t` and wanted to find a red clique of size `s` or a blue clique of size `t`. We now have `k` colours so we need a way of describing what size clique we would like for each of the possible colours. We can do this in Lean with `s : Fin k → ℕ`, so `s(c)` is the size of the clique we want to find in colour `c`.

Given `s : Fin k → ℕ` we say that a `k`-colouring `col : Finset ℕ → Fin k` is `kRamseyCol` for `n` and `s` iff for every finite subset of $\mathbb{N}$ of size at least `n`, there exists a subset `A` and a colour `c` such that all pairs in $\mathtt{A}^{(2)}$ are coloured with colour `c` and $\mathtt{s}(\mathtt{c}) \leq |\mathtt{A}|$.

```
def kRamseyCol (col : Finset ℕ → Fin k) (n : ℕ)
(s : Fin k → ℕ) : Prop := ∀ {V : Finset ℕ}, n ≤ |V| →
    ∃ A, ∃ c, A ⊆ V ∧ KmonoOn A col c ∧ s c ≤ |A|
```

Finally we can introduce `kRamseyOf N s`.

```
def kRamseyOf (N : ℕ) (s : Fin k → ℕ) : Prop :=
  ∀ (col : Finset ℕ → Fin k), kRamseyCol col N s
```

So `kRamseyOf N s` holds iff for *every* `k`-colouring, every subset of size at least `N` contains a set `A` such that for some colour `c`, $\mathtt{s}(\mathtt{c}) \leq |\mathtt{A}|$ and all pairs in $\mathtt{A}^{(2)}$ have colour `c`.

Ramsey's theorem for `k`-colours says that the `k`-colour Ramsey numbers are well-defined. This is equivalent to proving that for any `s : Fin k → ℕ` there exists an `N` such that `kRamseyOf N s` holds. (The Ramsey number is then defined to be the smallest such `N`.)

Our approach is to use induction and a "colour-blindness" argument.

Suppose the result holds for `k` and consider the case of `k + 1` colours. If `s : Fin (k + 1) → ℕ` describes the sizes of the `k + 1` possible cliques we wish to find then `kSizes s` is its restriction to the first `k` sizes.

```
def kSizes (s : Fin (k+1) → ℕ) : Fin k → ℕ := fun c => s c
```

The following lemma is the key inductive step.

```
lemma kRamsey_step (s : Fin (k+1) → ℕ)  :
    kRamseyOf a (kSizes s) → RamseyOf b a (s k) → kRamseyOf b s
```

We sketch the proof. Suppose that `a` satisfies `kRamseyOf a (kSizes s)` and `b` satisfies `RamseyOf b a (s k)`.

Given a `(k+1)`-colouring we convert this into a 2-colouring by considering all colours `< k` to be red and colour `k` to be blue. Applying Ramsey's theorem for 2-colours and using `RamseyOf b a (s k)` we either have a blue clique of size `s(k)` which corresponds to a clique of size `s(k)` and colour `k` in the original colouring; or we have a red clique of size `a` where `kRamseyOf a (kSizes s)` holds. Revealing the original colours in this clique (which are all $<$ `k`), the definitions of `kRamseyOf` and `kSizes` imply that it contains a clique of size `s(c)` coloured with colour `c` for some `c < k`.

The 2-colouring used in this sketch is defined as `kto2`.

```
def kto2 (col : Finset ℕ → Fin (k + 1)) : Finset ℕ → Fin 2 :=
    fun e => if col e = k then 1 else 0
```

The final definition we need converts a `(k + 1)`-colouring that only uses colours $<$ `k` on $\mathtt{A}^{(2)}$ into a `k`-colouring. (We used this implicitly in the second part of the sketch above.)

```
def kcol {col : Finset ℕ → Fin (k+1)}
 (hm : monoOn A (kto2 col) 0) : Finset ℕ → Fin k := by
  intro e
  by_cases he : e ∈ A.powersetLen 2
  · have hcol := hm e he
    use col e
    apply kto2_zero hcol
  · exact 0
```

Finally we have Ramsey's theorem for `k` colours.

```
theorem kcolour_Ramsey (s : Fin k → ℕ) : ∃ n, kRamseyOf n s
```

Possible extension: prove some constructive lower bounds for $k = 3$.

4.2.3 Schur's theorem (`Schur`)

Theorem (Schur)**.** *For any $k \in \mathbb{N}$ there exists $N \in \mathbb{N}$ such that in any k-colouring of $\mathbb{N}$, $c : \mathbb{N} \to \{1, 2, \ldots, k\}$ there exist $x, y, z \in \{1, \ldots, N\}$ such that $x + y = z$ and $c(x) = c(y) = c(z)$.*

Schur's theorem follows quite easily from the k-colour version of Ramsey's theorem which we will assume (see project `KColourRamsey`).

The key idea we will require is a way to convert between colourings of $\mathbb{N}$ and colourings of $\mathbb{N}^{(2)}$. We start by introducing the basic definitions.

We define `SchurOf n k` to hold iff for every `k`-colouring of $\mathbb{N}$ there is a colour `c` and `x, y` $\in \mathbb{N}$ such that `x, y` and `x + y` all receive colour `c`; `x, y > 0` and `x + y < n`.

```
def SchurOf (n k : ℕ): Prop := ∀ (Ncol : ℕ → Fin k), ∃ x y : ℕ,
 ∃ c : Fin k, Ncol x = c ∧ Ncol y = c ∧ Ncol (x + y) = c ∧
                  0 < x ∧ 0 < y ∧ x + y < n
```

The conversion between (`k + 1`)-colourings of $\mathbb{N}$ and $\mathbb{N}^{(2)}$ is accomplished as follows (note that this gives a (`k + 1`)-colouring of every finite subset of $\mathbb{N}$ but we will only consider the colour it assigns to pairs).

```
def Fcol (Ncol : ℕ → Fin (k + 1)) : Finset ℕ → Fin (k+1) :=
  fun A => if hne : A.Nonempty then Ncol (max' A hne - min' A hne)
                          else 0
```

We sketch the key argument: a monochromatic triangle in `Fcol` corresponds to a monochromatic Schur triple in the original colouring `Ncol`.

Let `Ncol : ℕ → Fin (k + 1)` be a (`k + 1`)-colouring of $\mathbb{N}$ and suppose that there exists monochromatic triangle in the converted colouring `Fcol`, i.e. a set `T = {u,v,w}` such that all three pairs `{u,v}`, `{v,w}` and `{v,w}` from `T` receive the same colour `c`. Assuming `u < v < w` and setting `x = v - u`, `y = w - v`, and `z = w - u` we can check that `x + y = z`, they are all positive, and in the original colouring of $\mathbb{N}$ they all receive the same colour `c`.

Hence using Ramsey's theorem for `k + 1` colours (and an appropriate choice of `n`) we can prove Schur's theorem.

```
theorem Schur (k : ℕ) : ∃ n : ℕ, SchurOf n k
```

Notice that the case `k` $= 0$ is vacuously true (although you will need to prove this separately since the conversion we sketched using `Fcol` is only defined for `k` $\neq 0$).

Possible extensions:

1. Prove some constructive lower bounds for small values of k.
2. Generalise the result to other equations.

4.2.4 Ramsey's theorem (hypergraph version) (RGraphKCol)

The aim of this project is to prove the hypergraph version of Ramsey's theorem. For any set X and $r \in \mathbb{N}$ let $X^{(r)}$ denote the family of all subsets of X of size r

$$X^{(r)} = \{A \subseteq X \mid |A| = r\}.$$

We will use `Fin 2 = {0,1}` as our colours (we will think of 0 as red and 1 as blue). A 2-colouring will be any map `col : Finset ℕ → Fin 2`. (Notice that our 2-colourings assign a colour to every finite subset of ℕ, this is simply to make things easier in the formalisation.)

If `A : Finset ℕ` then the Lean for $\mathtt{A}^{(\mathtt{r})}$ is `A.powersetLen r`.

A 2-colouring `col` is monochromatic with colour `c` on $\mathtt{A}^{(\mathtt{r})}$ iff the following holds. We will say simply $\mathtt{A}^{(\mathtt{r})}$ is red/blue.

```
def monoOnR (A : Finset ℕ) (r : ℕ) (col : Finset ℕ → Fin 2)
(c : Fin 2) : Prop := ∀ e ∈ A.powersetLen r, col e = c
```

Our key definition is that of `RamseyOf`.

```
def RamseyOf (N r s t : ℕ) : Prop :=
  ∀ col : Finset ℕ → Fin 2,
    ∀ V : Finset ℕ, N ≤ |V| → ∃ A : Finset ℕ, A ⊆ V ∧ (s ≤ |A|
     ∧ A.monoOnR r col 0 ∨ t ≤ |A| ∧ A.monoOnR r col 1)
```

Thus `RamseyOf N r s t` holds iff in *every* 2-colouring, every subset of size at least N contains a set `A` such that $|\mathtt{A}| \geq \mathtt{s}$ and $\mathtt{A}^{(\mathtt{r})}$ is red or $|\mathtt{A}| \geq \mathtt{t}$ and $\mathtt{A}^{(\mathtt{r})}$ is blue. Our main result is Ramsey's theorem for hypergraphs (and 2-colours).

```
theorem Ramsey_rgraphs (r s t : ℕ) : ∃ n : ℕ, RamseyOf n r s t
```

The proof follows a very similar inductive argument as that used in the `r = 2` version.

Rather than double induction we have triple induction starting with `r`. The following is the key lemma for the inductive step.

```
lemma Ramsey_step :  RamseyOf n r a b →
RamseyOf a (r + 1) s (t + 1) → RamseyOf b (r + 1) (s + 1) t
    → RamseyOf (n + 1) (r + 1) (s + 1) (t + 1)
```

The crucial definition for proving this lemma is the following which, for a fixed `x`, allows us to think about the 2-colouring of all the `(r + 1)`-sets containing `x` as a 2-colouring of `r`-sets. (Recall that `insert x e` is Lean for $\mathtt{e} \cup \{\mathtt{x}\}$).

```
def nbhd (x : ℕ) (col : Finset ℕ → Fin 2) :
    Finset ℕ → Fin 2 := fun e => col (insert x e)
```

We sketch the proof of the lemma. Suppose we know that `n`, `a` and `b` satisfy `RamseyOf n r a b`, `RamseyOf a (r + 1) s (t + 1)` and `RamseyOf b (r + 1) (s + 1) t`.

We want to show that `RamseyOf (n + 1) (r + 1) (s + 1) (t + 1)` holds. Take a 2-colouring `col` and a set $\mathtt{V} \subseteq \mathbb{N}$ of size at least `n + 1` and remove an element `x`. Now consider the 2-colouring given by `nbhd x col` inside `V`. Since `RamseyOf n r a b` holds we can find a set $\mathtt{A} \subseteq \mathtt{V}$ such that either $\mathtt{a} \leq |\mathtt{A}|$ and $\mathtt{A}^{(\mathtt{r})}$ is red or $\mathtt{b} \leq |\mathtt{A}|$ and $\mathtt{A}^{(\mathtt{r})}$ is blue, both in the `nbhd` colouring. In the former case, since `RamseyOf a (r + 1) s (t + 1)` holds, the set `A` contains a set `C` such that either $\mathtt{s} \leq |\mathtt{C}|$ and $\mathtt{C}^{(\mathtt{r}+1)}$ is red or $\mathtt{t} + 1 \leq |\mathtt{C}|$ and $\mathtt{C}^{(\mathtt{r}+1)}$ is blue (both in the original colouring). In the latter case we are done, in the former we can check that $\mathtt{s} + 1 \leq |\mathtt{C} \cup \{\mathtt{x}\}|$ and $(\mathtt{C} \cup \{\mathtt{x}\})^{(r+1)}$ is red, in the original colouring. The other case, with $\mathtt{b} \leq |\mathtt{A}|$ and $\mathtt{A}^{(\mathtt{r})}$ is blue in the `nbhd` colouring, follows similarly using the assumption that `RamseyOf b (r + 1) (s + 1) t`.

The other definitions we require relate to the obvious symmetry between colours red 0 and blue 1.

```
def notC (c : Fin 2) : Fin 2 := if (c = 0) then 1 else 0

def notCol (col : Finset ℕ → Fin 2) :
        Finset ℕ → Fin 2 :=  notC ∘ col
```

Possible extension: prove the same result for any number of colours `k`. The definitions are given below. See the project (`KColourRamsey`) for details of how the proof goes for $\mathtt{r} = 2$, these are almost unchanged in the general case.

```
def KmonoOnR (A : Finset ℕ) (r : ℕ)  (col : Finset ℕ → Fin k)
(c : Fin k) : Prop :=
    ∀ e : Finset ℕ, e ∈ A.powersetLen r  → col e = c

def KRamseyOf (n r : ℕ) (s : Fin k → ℕ) : Prop :=
  ∀ col : Finset ℕ → Fin k,
    ∀ {V : Finset ℕ}, n ≤ |V| → ∃ A : Finset ℕ, ∃ c : Fin k,
    A ⊆ V ∧ s c ≤ |A| ∧  A.KmonoOnR r col c

def kSizes (s : Fin (k+1) → ℕ) : Fin k → ℕ := fun c => s c

def kto2 (col : Finset ℕ → Fin (k+1)) : Finset ℕ → Fin 2 :=
  fun e => if col e < Fin.last k then 0 else 1

def kcol {col : Finset ℕ → Fin (k+1)}
(hm : monoOnR A r (kto2 col) 0) : Finset ℕ → Fin k := by
  intro e
  by_cases he : e ∈ A.powersetLen r
  · have hcol := hm e he
    use col e
    apply kto2_zero hcol
  · exact 0
```

5 Linear Algebra (LinearAlgebra)

5.1 Steinitz Exchange Lemma (SteinitzExchange)

The main goal of this project is to prove the Steinitz Exchange lemma and deduce that any two bases in a finite dimensional real vector space have the same size.

We start by defining a class VectorSpace.

```
class VectorSpace (V : Type) extends AddCommGroup V,
    DistribMulAction ℝ V where
  add_smul (x y : ℝ) (v : V) : (x + y) • v = x • v + y • v
  zero_smul (v : V) : (0 : ℝ) • v = 0
```

We then define the *linear combination* of a finite set of vectors A; its *span*; the proposition that A *Spans* V; that A is *dependent*; *independent*; a basis and that V is finite-dimensional.

```
def linearCombination (A : Finset V) (c : V → ℝ) :
    V := Σ a in A, c a • a
def span (A : Finset V) : Set V := range (linearCombination A)
def Spans (A : Finset V) : Prop :=
∀ (v : V), ∃ c : V → ℝ , linearCombination A c = v
def Dependent (A : Finset V) : Prop :=
  ∃ c : V → ℝ, linearCombination A c = 0 ∧  ∃ a ∈ A, c a ≠ 0
def Independent (A : Finset V) : Prop := ¬ Dependent A
def IsBasis (A : Finset V) : Prop := Spans A ∧ Independent A
def FiniteDim : Prop := ∃ A : Finset V, Spans A
```

Our main result is the following.

Theorem (Steinitz)**.** *If A is a finite independent set and and B is a finite spanning set in a vector space V, then there exists a spanning set $A \cup C$ with $C \subseteq B$ and $|A \cup C| = |B|$.*

As a corollary we obtain the following result.

Theorem. *If A and B are bases of a finite dimensional vector space V then $|A| = |B|$.*

There are many possible extensions, we list some examples.

Define the external direct sum `V ⊕ W` as a vector space and prove that

```
dim V ⊕ W = dim V + dim W
```

Define a structure `Subspace V` and construct `span A` as a term of type `Subspace V`.

Define a structure `Linear V W` of linear maps from `V` to `W`.

Construct the image and kernel of a linear map as a subspace. Prove the rank-nullity theorem.

Define an instance of `Add (Subspace V)` (the sum of two subspaces).

Prove that addition of subspaces is commutative and associative.

Define the intersection of two subspaces as a subspace.

Prove the dimension formula:

```
dim (A + B) + dim (A ∩ B) = dim A + dim B
```

5.2 Elementary matrices and row operations (`ElemMatRow`)

Establish basic facts about elementary matrices and correspondence between left-multiplication and row operations.

We introduce four matrix row operations. The first three are the standard ones: add a scalar multiple of row j onto row i; swap rows i and j; multiply row i by a scalar.

```
def add_mul_row (M : Matrix (Fin m) (Fin n) 𝔽) (i j : Fin m)
(c : 𝔽) : Matrix (Fin m) (Fin n) 𝔽:=
  M.updateRow i (M i + (fun l => c * M j l))

def swap (M : Matrix (Fin m) (Fin n) 𝔽) (i j : Fin m): Matrix (
    Fin m) (Fin n) 𝔽:=
 (M.updateRow i (M j)).updateRow j (M i)

def mul_row (M : Matrix (Fin m) (Fin n) 𝔽) (i : Fin m) (c : 𝔽):
  Matrix (Fin m) (Fin n) 𝔽:=  M.updateRow i (c • (M i))
```

The fourth row operation we introduce is just a special case of the first, but it plays a crucial role in Gaussian elimination. Subtract $M_{i,j}M_{j,j}^{-1}$ times row j from row i. (Note that if $M_{j,j} \neq 0$ this operation will produce a zero at entry (i, j)).

```
def sub_mul_inv (M : Matrix (Fin n) (Fin n) 𝔽) (i j : Fin n) :
 Matrix (Fin n) (Fin n) 𝔽:=
  M.updateRow i (M i - (fun l => (M i j)* (M j j)⁻¹*(M j l)))
```

We also introduce definitions for corresponding elementary matrices in terms of the `stdBasisMatrix i j a` which is the matrix with zeros everywhere except entry (i, j) which equals `a`.

```
def AddMul (i j : Fin m) (c : 𝔽) : Matrix (Fin m) (Fin m) 𝔽 :=
    1 + stdBasisMatrix i j c

def Swap (i j : Fin m) : Matrix (Fin m) (Fin m) 𝔽:=
    1 + (stdBasisMatrix i j 1) + (stdBasisMatrix j i 1) -
    (stdBasisMatrix i i 1) - (stdBasisMatrix j j 1)

def MulRow (i : Fin m) (c : 𝔽) : Matrix (Fin m) (Fin m) 𝔽 :=
    1 + stdBasisMatrix i i (c - 1)
```

Theorem. *Each row operation is equivalent to left multiplication by the corresponding matrix.*

Theorem. *These elementary matrices are all invertible (except `MulRow i 0`).*

Possible extension: start the Gaussian Elimination project making use of these results about elementary matrices.

5.3 Gaussian Elimination (`GaussElim`)

Given an $n \times n$ matrix M over a field $\mathbb{F}$ we will consider the computational problem of deciding whether or not M is invertible, computing its inverse if it exists and computing a non-trivial solution x to $Mx = 0$ if it is not.

We describe the (easy) mathematics first.

Our main algorithm (which we will call `GaussStep` in Lean) is as follows. Our input is an $n \times n$ matrix M over a field $\mathbb{F}$. We will apply elementary row operations to put M into upper triangular form (i.e. so that all entries below the diagonal are zero).

If M is upper triangular then there is nothing to do so suppose it is not. Identify the first column containing a non-zero entry below the diagonal and call this j. Now identify the first non-zero entry in column j below the diagonal and call its row index i. We call the position (i, j) the *pivot*. We now apply one of two possible row operations depending on whether or not $M_{j,j} = 0$. If $M_{j,j} = 0$ we swap rows i and j, otherwise we subtract $M_{i,j}M_{j,j}^{-1}$ times row j from row i. Repeat until the matrix is in upper triangular form.

[The project `GaussStep` below involves proving that this does indeed terminate with an upper triangular matrix in at most $n(n+1)$ steps.]

See below for an example of applying this algorithm, at each step the pivot is underlined.

$$\begin{pmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ \underline{1} & 3 & 3 & 0 \\ 2 & 4 & 1 & 2 \end{pmatrix} \quad \begin{pmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 1 & 2 & 0 \\ \underline{2} & 4 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & \underline{1} & 2 & 0 \\ 0 & 0 & -1 & 2 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & \underline{-1} & 2 \end{pmatrix} \quad \begin{pmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

If this upper triangular form (which we will call `M.GaussUt` in Lean) has no zeros on its diagonal (such as in our example) then M is invertible and we compute its inverse as follows. Form the transpose of `M.GaussUt` and apply the `GaussStep` algorithm again. See below for the continuation of our example.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ \underline{2} & 1 & 0 & 0 \\ 1 & 2 & -1 & 0 \\ 0 & 1 & 2 & -1 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \underline{1} & 2 & -1 & 0 \\ 0 & 1 & 2 & -1 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & \underline{2} & -1 & 0 \\ 0 & 1 & 2 & -1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & \underline{1} & 2 & -1 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & \underline{2} & -1 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

As we will prove (and is easy to see), this is guaranteed to produce a diagonal matrix with no zero diagonal entries. Such a matrix is easy to invert. Moreover, by using the equivalence between row operations and left multiplication by elementary matrices, we can hence compute the inverse of M.

[The project `Inverse` below involves proving that we can indeed compute the inverse of M in this way.]

We now consider the slightly more complicated case when the upper triangular form `M.GaussUt` has a zero on its diagonal. In this case we want to prove that M is not invertible by computing a non-trivial solution to $Mx = 0$.

Let `G = M.GaussUt` be the upper triangular matrix obtained by applying `GaussStep` to M. So, by assumption `G` has a zero on its diagonal. Let `k` be the index of the first such zero. Form a block matrix B which agrees with G in the top left $k \times k$ block and otherwise agrees with the identity matrix

$$B = \begin{pmatrix} G_k & 0 \\ 0 & I_{n-k} \end{pmatrix}.$$

Since `k` was the index of the first zero on the diagonal of G (and our columns are indexed from 0) so G_k is upper triangular with no zeros on its diagonal. Hence B is upper triangular with no zeros on its diagonal and thus invertible. Moreover if its inverse is N then NG agrees with the identity matrix on its first k columns while its $(k+1)$th column (which is indexed by `k`) is zero from row `k` onwards. Using this information we can check that the column vector x defined below satisfies $(NG)x = 0$:

$$x_i = \begin{cases} -(NG)_{i,k}, & i < k, \\ 1, & i = k, \\ 0, & k < i. \end{cases}$$

Since N is invertible and $G = UM$, where U is the invertible product of elementary matrices that puts M into upper triangular form, we see that x is a non-trivial zero for our original matrix M. (The non-triviality of x follows from the fact that $x_k = 1 \neq 0$).

[The project `NotInvertible` below involves proving that this method does indeed work.]

We illustrate this argument with an example. If

$$M = \begin{pmatrix} 1 & 2 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 2 & 2 & 5 \\ 1 & 2 & 1 & 3 \end{pmatrix} \quad \text{then} \quad G = \begin{pmatrix} 1 & 2 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 2 \end{pmatrix}.$$

So the first zero on the diagonal of G is at position $k = 2$. Hence we form the block matrix

$$B = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{with inverse} \quad N = \begin{pmatrix} 1 & -2 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

We compute the product NG and x as described above.

$$NG = \begin{pmatrix} 1 & 0 & -1 & -3 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad x = \begin{pmatrix} 1 \\ -1 \\ 1 \\ 0 \end{pmatrix}$$

Finally we check that x is indeed a non-trivial zero of M

$$Mx = \begin{pmatrix} 1 & 2 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 0 & 2 & 2 & 5 \\ 1 & 2 & 1 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

5.3.1 Reduction to upper triangular form (`GaussStep`)

A matrix is *upper triangular* iff every element below the leading diagonal is zero.

```
def Matrix.ut (M : Matrix (Fin n) (Fin n) 𝔽) : Prop :=
    ∀ i j, j < i → M i j = 0
```

In this project we consider the first part of Gaussian elimination: given an $n \times n$ matrix over a field $\mathbb{F}$ apply row operations to obtain an upper triangular matrix. Our main goal is to define this algorithm and prove that it successfully terminates in at most $n(n+1)$ steps.

Our approach follows the algorithm `GaussStep` described in the previous section.

We will be working mainly with matrices that are not upper triangular, hence we introduce the following definitions.

```
def Matrix.nut (M : Matrix (Fin n) (Fin n) 𝔽) : Prop :=
    ∃ i j, j < i ∧ M i j ≠ 0

structure NUT (n : ℕ) (𝔽 : Type) [Field 𝔽] where
mat : Matrix (Fin n) (Fin n) 𝔽
nut' : mat.nut
```

If `M` is a not upper triangular matrix then there is a 1st column J such that `M` contains a non-zero entry below the diagonal in column J. We want to define this index J computably so we do it as follows.

```
def J (M : NUT n 𝔽) : Fin n := by
  let K :=(univ : Finset (Fin n)).filter
            (fun k => ∃ i, k < i ∧ M.mat i k ≠ 0)
  exact K.min' M.non_zero_cols_nonempty
```

We can then identify the row index of the first non-zero entry below the diagonal in column J and we call this I. The position (I, J) is the *pivot*.

```
def I (M : NUT n 𝔽) : Fin n := by
  let L : Finset (Fin n):= (univ : Finset (Fin n)).filter
                (fun l => M.J < l ∧ M.mat l M.J ≠ 0)
  exact L.min' M.non_zero_entry_of_non_zero_col
```

We also introduce a weight function for not upper triangular matrices.

```
def weight (M : NUT n 𝔽) : ℕ := M.I + n * M.J
```

We prove some basic properties of this weight function that will allow us to later prove that `GaussStep` terminates.

Lemma. *If M is a not upper triangular matrix and $M_{J,J} = 0$ then swapping rows I and J increases the weight.*

Lemma. *If M is a not upper triangular matrix and $M_{J,J} \neq 0$ then subtracting the appropriate multiple of row J from row I increases the weight.*

We can now define the `GaussStep` algorithm formally: it will swap rows I and J (if $M_{J,J} = 0$) or subtract an appropriate multiple of row J from row I otherwise.

```
def GaussStep (M : Matrix (Fin n) (Fin n) 𝔽) :
 Matrix (Fin n) (Fin n) 𝔽 := by
  by_cases hnut: M.nut
  · let N : NUT n 𝔽 := ⟨M,hnut⟩
    let i := N.I
    let j := N.J
    exact if M j j = 0 then M.swap i j else (M.sub_mul_inv i j)
  · exact M
```

Our original `weight` function was only defined for not upper triangular matrices, but we will want to have a weight function defined for all $n \times n$ matrices. We do this as follows.

```
def weight' (M : Matrix (Fin n) (Fin n) 𝔽) : ℕ :=
if h : M.nut then (n * (n + 1) + 1 - (⟨M, h⟩ : NUT n 𝔽).weight)
             else 0
```

Note that `weight'` now decreases as we apply a `GaussStep` and that it equals zero iff we have an upper triangular matrix.

Our main theorem is as follows.

Theorem. *Gaussian elimination (in the form of iterated applications of `GaussStep`) applied to an $n \times n$ matrix over a field $\mathbb{F}$ will terminate with an upper triangular matrix after at most $n(n+1)$ steps.*

Possible extensions: prove a better upper bound on the number of steps required before `GaussStep` terminates.

Extend the algorithm to rectangular matrices and row echelon form.

5.3.2 Inverting an invertible matrix (`Inverse`)

Prove that we can use the `GaussStep` algorithm introduced in the previous section to compute the inverse of a matrix (if the matrix is invertible).

The basic new definitions we will need include the following predicates for matrices: invertible; lower triangular; no zero on the diagonal; is diagonal.

```
def Matrix.invertible (M : Matrix (Fin n) (Fin n) 𝔽) : Prop :=
    ∃ N, M * N = 1 ∧ N * M = 1

def Matrix.lwt (M : Matrix (Fin n) (Fin n) 𝔽) : Prop := ut M^T

def Matrix.no_zero_on_diag (M : Matrix (Fin n) (Fin n) 𝔽) :
    Prop := ∀ i, M i i ≠ 0

def Matrix.is_diagonal (M : Matrix (Fin n) (Fin n) 𝔽) :
    Prop := ∀ i j, i ≠ j → M i j = 0
```

We start from the theorem proved in the previous project, which tells us that given any $n \times n$ matrix `M` over a field $\mathbb{F}$ we have a function `GaussStep` which, when iterated $n(n+1)$ times, produces an upper triangular matrix.

```
theorem upperbound_on_steps (M : Matrix (Fin n) (Fin n) 𝔽) :
    (GaussStep^[n*(n+1)] M).ut
```

Our algorithm for inverting `M` proceeds as follows. Given `M` first compute its upper triangular form.

```
def Matrix.GaussUt (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := GaussStep^[n*(n+1)] M
```

Note that although we will not prove this here (see the next project) `M` is invertible iff this upper triangular form has no zeros on its diagonal.

Thus our computation of the inverse of `M` will stop at this point and output 0 if there are any zeros on the diagonal of `M.GaussUt`. For the rest of the proof we will suppose that this does not occur (and part of the proof will consist of checking that if `M.GaussUt` has no zeros on its diagonal then this remains true of the matrices we obtain as we continue this process).

Take the transpose so we have a lower triangular matrix: `M.GaussUt`^{T}.

We now apply the `GaussStep` algorithm to this matrix to obtain a diagonal matrix (you will need to prove that it is indeed diagonal).

```
def Matrix.GaussDiag (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := M.GaussUt^T.GaussUt
```

Recall that inverting (invertible) diagonal matrices is easy.

```
def Matrix.InverseDiag (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := diagonal (fun i => (M i i)⁻¹)
```

Computing the inverse of `M` now simply requires us to translate the row operations performed by `GaussStep` into equivalent multiplications by elementary matrices. We start by defining these for a single step.

```
def GaussStepMat (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := by
  by_cases hnut: M.nut
  · let M′ : NUT n 𝔽 := ⟨M,hnut⟩
    let i := M′.I
    let j := M′.J
    exact if M j j = 0 then Swap i j
            else AddMul i j (-(M i j)*(M j j)⁻¹)
  · exact 1
```

We then define the iterated version.

```
def Matrix.GaussStepIterMat (M : Matrix (Fin n) (Fin n) 𝔽)
 (k : ℕ) : Matrix (Fin n) (Fin n) 𝔽 := by
   induction k with
   | zero => exact 1
   | succ k Mk => exact (GaussStepMat (GaussStep^[k] M)) * Mk
```

In particular we have the matrices that put `M` into upper triangular and then diagonal form.

```
def Matrix.U (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := M.GaussStepIterMat (n*(n+1))

def Matrix.L (M : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) (Fin n) 𝔽 := (M.U * M)ᵀ.U
```

These satisfy `M.GaussDiag = M.L * M`^{T} `* M.U`^{T}

We can then compute the inverse of `M` as follows.

```
 def Matrix.Inverse  (M : Matrix (Fin n) (Fin n) 𝔽) :
 Matrix (Fin n) (Fin n) 𝔽 :=
  if M.GaussUt.no_zero_on_diag then
    (M.Lᵀ * M.GaussDiag.InverseDiag * M.U) else 0
```

Our main result now says the following.

Theorem. *If the upper triangular form `M.GaussUt`, has no zeros on its diagonal then the matrix output by `M.Inverse` is indeed the inverse of `M`.*

Possible extension: do the project `NotInvertible` to deal with the case when `M` is not invertible.

5.3.3 Non-invertible (`NotInvertible`)

Prove that an $n \times n$ matrix `M` is invertible iff its upper triangular form `M.GaussUt` given by applying the `GaussStep` algorithm has no zeros on its leading diagonal.

In the previous project we computed the inverse of `M` in the case that the upper triangular form `M.GaussUt` has no zeros on its diagonal, thus the main thing we need to prove here is that if there is a zero on the diagonal of `M.GaussUt` then the matrix `M` is not invertible.

We first prove that if a matrix `M` is invertible then the only solution to `M * B = 0` is `B = 0`. The remainder of the proof is essentially a computation of a non-trivial zero for `M`.

Given any $n \times n$ matrix M and any k we define a block matrix `M.Block k` that agrees with M in the upper-left $k \times k$ block and agrees with the identity matrix everywhere else.

```
def Matrix.Block (M : Matrix (Fin n) (Fin n) 𝔽) (k : Fin n) :
  Matrix (Fin n) (Fin n) 𝔽 := Matrix.of
    (fun i j => if (j < k) then (M i j)
            else (if (i = j) then 1 else 0))
```

Let `G = M.GaussUt` be the upper triangular matrix obtained by applying `GaussStep` to `M`. So, by assumption `G` has a zero on its diagonal. Let `k` be the index of the first such zero. We now consider the block matrix:

$$\texttt{G.Block k} = \begin{pmatrix} G_k & 0 \\ 0 & I_{n-k} \end{pmatrix}$$

Note that since columns are indexed from zero (in `Fin n`) so G_k is upper triangular with no zeros on its diagonal. Hence `G.Block k` is invertible. Moreover if its inverse is

$$N = \begin{pmatrix} G_k & 0 \\ 0 & I_{n-k} \end{pmatrix}^{-1}$$

then NG agrees with the identity matrix on its first k columns while its kth column is zero from row k onwards. Using this information we can check that the (non-zero) column vector x defined below satisfies $(NG)x = 0$:

$$x_i = \begin{cases} -(NG)_{i,k}, & i < k, \\ 1, & i = k, \\ 0, & k < i. \end{cases}$$

This in turn implies that $Mx = 0$ as required (since N is invertible and $G = UM$, with U invertible).

To formalise this argument we introduce a function computing the index of the first zero on the diagonal of a matrix.

```
def Matrix.fz (M : Matrix (Fin n) (Fin n) 𝔽) : Fin n :=
by
```

```
by_cases hninv : M.no_zero_on_diag
 · exact 0
 · let _Z : Finset (Fin n) := (univ : Finset (Fin n)).filter
                                (fun i =>  M i i = 0)
   rw [no_zero_on_diag] at hninv; push_neg at hninv
   have Zne: _Z.Nonempty
   · obtain ⟨i,hiz⟩:=hninv
     refine ⟨i, mem_filter.2 ⟨mem_univ _ ,hiz⟩⟩
   exact _Z.min' Zne
```

Now the non-trivial zero x described above can be computed as follows:

```
def Matrix.x (G : Matrix (Fin n) (Fin n) 𝔽) :
    Matrix (Fin n) Unit 𝔽 := (fun i  _ => if i < G.fz then
        -((G.Block G.fz).Inverse * G) i G.fz
             else (if i = G.fz then 1 else 0))
```

Finally we prove that if `M.GaussUt` has a zero on its diagonal then setting `G= M.GaussUt` the vector `G.x = M.GaussUt.x` is a non-trivial zero of `M` and hence `M` is not invertible.

Combining this result with the main theorem of the previous project (that if `M.GaussUt` has no zeros on its diagonal then `M` is invertible) we obtain the following theorem.

Theorem. *A matrix is invertible iff the upper-triangular matrix `M.GaussUt` has no zeros on its diagonal.*

Possible extension: give some examples of deciding invertibility of small matrices using the code in this project.

6 Number Theory (`NumberTheory`)

6.1 Convergence of continued fractions (`ContinuedFrac`)

This is a topic from MATH0034.

Let a_n be a sequence of positive integers. Show that the continued fraction converges:

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \ddots}}}.$$

In order to do this, we write the n-th convergent in the form partQuot n = numer n/denom n, for certain recursively defined sequences numer and denom. Then prove that

$$\text{partQuot } (n+1) = \text{partQuot } n + \frac{(-1)^n}{\text{denom } n * \text{denom } (n+1)}.$$

Using this, you should be able to prove convergence of the sequence partQuot n.

As an extension, you could prove that every real number has a continued fraction expansion. You could then prove a formula for the continued fraction expansion of $\sqrt{d}$ for some positive integer d.

6.2 Divisors (`Divisors`)

In this project we formalise the following simple result.

Theorem. *If $S \subseteq \{1, \ldots, 2n\}$ with $n \geq 1$ and $|S| \geq n + 1$ then $\exists a, b \in S$ with $a \neq b$ such that $a \mid b$.*

The basic idea of the proof is to define a function mapping any $a \in \mathbb{N}$ to its maximum odd factor. If $a, b \in S \subseteq \{1, \ldots, 2n\}$ have the same maximum odd factor m then there exist $i, j \in \mathbb{N}$ such that $a = 2^i m$ and $b = 2^j m$ and hence $a \mid b$ or $b \mid a$. Since the maximum odd factor of $a \in \{1, \ldots, 2n\}$ is odd it can take one of at most n distinct values: $\{1, 3, \ldots, 2n - 1\}$ and the result follows.

In order to formalise this in Lean we introduce introduce two definitions.

```
def Divisible (S : Finset ℕ) : Prop :=
            ∃ a ∈ S, ∃ b ∈ S, a ≠ b ∧ a ∣ b
```

Given `n : ℕ` and a prime `p`, Lean has a function `ord_proj[p] n` which returns the highest power of `p` in the prime factorisation of `n`.

Lean also defines `ord_compl[p] n` to be `n / ord_proj[p] n`. Thus we can define the maximum odd factor of `n` as follows.

```
def max_odd_factor : ℕ → ℕ := fun n => ord_compl[2] n
```

After proving the theorem we end by giving an example to show that this theorem is best possible.

```
theorem exists_not_div_card_eq : ∃ S, S ⊆ Ioc 0 (2 * n) ∧
    ¬Divisible S ∧ S.card = n
```

Possible extension: consider $S \subseteq \{1, ..., 2^{k+1}n\}$, where $k \geq 1$. Prove that if $(2^{k+1} - 1)n < |S|$ then S contains $k + 2$ pairwise divisors, i.e. $D \subseteq S$ such that $k + 2 \leq |D|$ and $\forall a, b \in D$, $a \mid b$ or $b \mid a$.

6.3 The Eisenstein integers (`EisensteinIntegers`)

Construct the ring of Eisenstein integers and prove that this ring is norm-Euclidean.

Let $\omega = \frac{1+\sqrt{-3}}{2}$. An Eisenstein integer is a complex number of the form $x + y\omega$, where x and y are integers. The set of all of these numbers is written $\mathbb{Z}[\omega]$. We can define the Eisenstein integers as a structure as follows:

```
structure EisensteinInt : Type where
  x : ℤ
  y : ℤ
notation "ℤ[ω]" => EisensteinInt
```

- Define operations of addition and multiplication on the Eisenstein integers and prove that they form a commutative ring. I.e. create an instance

  ```
  instance : CommRing ℤ[ω] := sorry
  ```

- Define a function which takes an Eisenstein integer to the corresponding complex number. Prove that this function is a ring homomorphism.
- Define the norm function $N(x + y\omega) = x^2 - xy + y^2$. Prove that $N(A) = |A|^2$.
- Prove that if $A, B \in \mathbb{Z}[\omega]$ with $B \neq 0$, then there exist $Q, R \in \mathbb{Z}[\omega]$, such that $A = QB + R$ and $N(R) < N(B)$. Using this lemma, create an instance

  ```
  instance : EuclideanDomain ℤ[ω] := sorry
  ```

The key lemma above is the following:

Lemma. *Let $A, B \in \mathbb{Z}[\omega]$ with $B \neq 0$, then there exist $Q, R \in \mathbb{Z}[\omega]$, such that $A = QB + R$ and $N(R) < N(B)$.*

Proof. Let $\frac{A}{B} = x + y\omega$ with $x, y \in \mathbb{Q}$. Choose the nearest integer b to y, so that $|b - y| \leq \frac{1}{2}$. This implies $|\Im(x + (y - b)\omega)| \leq \frac{\sqrt{3}}{4}$. Then choose the nearest integer a to $x + (y - b)\omega$. Then we have $|\Re((x - a) + (y - b)\omega| \leq \frac{1}{2}$. As a consequence of this, we have

$$N\left(\frac{A}{B} - (a + b\omega)\right) = |((x - a) + (y - b)\omega|^2 \leq \left(\frac{1}{2}\right)^2 + \left(\frac{\sqrt{3}}{4}\right)^2 < 1.$$

If we set $Q = a + b\omega$ and $R = A - QB$, then we have $A = QB + R$. We have from the inequality above:

$$N\left(\frac{R}{B}\right) = N\left(\frac{A}{B} - (a + b\omega)\right) < 1,$$

and this implies $N(R) < N(B)$. $\square$

Possible extensions:

- In the lemma above, define the quotient Q and the remainder R as computable functions of A and B. Then define a function $\gcd(A, B)$ recursively by
$$\gcd(A, B) = \begin{cases} A & B = 0 \\ \gcd(B, R(A, B)) & B \neq 0 \end{cases}$$
Prove that $\gcd(A, B)$ is a common factor of A and B with smallest possible norm.

- generalize the project above to prove the "Disappointing Theorem" from MATH0034, which says that $\mathbb{Z}[\frac{1+\sqrt{d}}{2}]$ is a Euclidean ring for $d = -3, -7, -11, 5, 13$ and $\mathbb{Z}[\sqrt{d}]$ is a Euclidean ring for $d = -1, -2, 2$.

6.4 Infinitude of primes (`Furstenberg`)

Formalise Furstenberg's topological proof that there exist infinitely many primes. (This only requires the most basic facts about topological spaces.)

For any $a, b \in \mathbb{N}$ we define the associated arithmetic progression:

```
def AP (a b : ℕ) : Set ℕ := {n : ℕ | ∃ k : ℕ, n = a * k + b}
```

Before introducing a topology on $\mathbb{N}$ we establish two main results.

Lemma. *The union of the arithmetic progressions of the form* $\mathtt{AP}(p, 0)$*, where* p *is prime satisfies:*

$$\mathbb{N} \setminus \{1\} = \bigcup_{p \text{ prime}} \mathtt{AP}(p, 0).$$

Lemma. *If* $a, b \in \mathbb{N}$ *and* $a < b$ *then we can express the arithmetic progression* $\mathtt{AP}(a, b)$ *as the complement of a union of arithmetic progressions:*

$$\mathtt{AP}(a, b) = \mathbb{N} \setminus \bigcup_{j<a,\ j \neq b} \mathtt{AP}(a, j).$$

A *topology* τ on a set X is a collection of subsets of X satisfying:

(1) $X \in \tau$,

(2) $U, V \in \tau \implies U \cap V \in \tau$,

(3) If $\sigma \subseteq \tau$ then $\bigcup_{U \in \sigma} U \in \tau$.

Note that by taking σ to be the empty subfamily in (3) we always have $\emptyset \in \tau$.

We call the members of τ the *open* sets of the topological space (X, τ). The properties can be summarised as: (1) the whole space is open; (2) pairwise intersections of open sets are open; (3) arbitrary unions of open sets are open.

Given (X, τ), a topological space, we say that $F \subseteq X$ is *closed* iff $X \setminus F$ is open.

We will define the *Furstenberg topology* on $\mathbb{N}$ using the following definition.

```
def Fopen : Set ℕ → Prop :=
    fun U => ∀ n ∈ U, ∃ a, a ≠ 0 ∧ AP a n ⊆ U
```

We prove next that this does indeed define a topology on $\mathbb{N}$.

Lemma. *The Furstenberg topology is indeed a topology on* $\mathbb{N}$*.*

We establish some simple facts about arithmetic progressions in this topology.

Lemma. *For any* $a, b \in \mathbb{N}$*, if* $a \neq 0$ *then* $\mathtt{AP}(a, b)$ *is open.*

Lemma. *For any* $a, b \in \mathbb{N}$*, if* $b < a$ *then* $\mathtt{AP}(a, b)$ *is closed.*

We now consider whether $\{1\}$ is open in this topology and prove the key result.

Theorem. *If there are only finitely many primes then* $\{1\}$ *is an open set.*

But we can easily prove that $\{1\}$ is not open.

Lemma. *The set* $\{1\}$ *is not open.*

Corollary. *There exist infinitely many primes.*

Possible extension: explore the topological properties of $\mathbb{N}$ (with the Furstenberg topology). For example prove that it is a Hausdorff space.

6.5 Units in the Ring $\mathbb{Z}[\frac{1+\sqrt{5}}{2}]$ (`QuadraticRingUnits`)

Construct the ring $\mathbb{Z}[\frac{1+\sqrt{5}}{2}]$ and describe the units.

Let $\alpha = \frac{1+\sqrt{5}}{2}$. The set of all numbers $x+y\alpha$ with $x, y \in \mathbb{Z}$ is written $\mathbb{Z}[\alpha]$. This set is a ring with the usual operations of addition and multiplication. Construct this ring as a structure. Prove that α is a unit in the ring, and prove that every unit has the form $\pm\alpha^n$ for some $n \in \mathbb{Z}$.

- Create an instance

```
instance : CommRing ℤ[α] := sorry
```

- Define a function which takes an Eisenstein integer to the corresponding real number. Prove that this function is a ring homomorphism. Define another function which takes $x + y\alpha$ to $x + y(1 - \alpha)$, and show that this is also a ring homomorphism.
- Define the norm function $N(x+y\alpha) = x^2+xy-y^2$. Prove that $N(AB) = N(A)N(B)$.
- Prove that an element $A \in \mathbb{Z}[\alpha]$ is a unit if and only if $N(A) = \pm 1$.
- Prove that α is a unit.
- Prove that every unit in $\mathbb{Z}[\alpha]$ is $\pm\alpha^n$ for some $n \in \mathbb{Z}$.

The key lemmas above are the following:

Lemma. *Let F_n be the Fibonacci sequence $0, 1, 1, 2, 3, 5, \ldots$. Then $\alpha^{n+1} = F_n + F_{n+1}\alpha$.*

Proof. This is an induction proof using the relation $\alpha^2 = \alpha + 1$. □

Conversely we have:

Lemma. *Suppose $x, y \in \mathbb{N}$ with $y > 0$ and $N(x + y\alpha) = x^2 + xy - y^2 = \pm 1$. Then $x = F_n$ and $y = F_{n+1}$ are consecutive Fibonacci numbers.*

Proof. We'll prove the lemma by strong induction on y. If $y = 1$ then $x = 0$ or $x = 1$. The result holds in either case.

Assume $y \geq 2$. Clearly $x < y$, since otherwise $x^2 + xy - y^2 \geq x^2 \geq 4$. Using this, we can check that $(y - x, x)$ is a smaller solution to the equation. By the inductive hypothesis, we have for some natural number n,

$$y - x = F_n, \qquad x = F_{n+1}.$$

This implies $y = (y - x) + x = F_n + F_{n+1} = F_{n+2}$. □

As a consequence we can prove the following:

Theorem. *If u is an unit in $\mathbb{Z}[\alpha]$ then $u = \pm\alpha^n$ for some $n \in \mathbb{Z}$.*

Proof. Let $u = x + y\alpha$. Replacing u by $-u$ or u^{-1} or $-u^{-1}$ if necessary, we can assume that $u \geq 1$. This implies $x, y \in \mathbb{N}$ and either $u = 1$ or $y \geq 1$. The result then follows using the previous two lemmas. $\square$

Possible extension: Prove the that ring $\mathbb{Z}[\alpha]$ is Norm-Euclidean.

6.6 Sum of reciprocals of primes (`SumRecipPrimes`)

Theorem. *If $m \in \mathbb{N}$ then the sum of the reciprocals of the primes less than $2^{4m}+1$ satisfies*

$$\log(m+1/2) \leq \sum_{p<2^{4m}+1} \frac{1}{p}.$$

Let $f : \mathbb{N} \to \mathbb{R}$ be a `MonoidWithZeroHom`, i.e. $f(0) = 0$, $f(1) = 1$ and $\forall a, b$, $f(ab) = f(a)f(b)$. Suppose further that f is non-negative, i.e. $\forall k, f(k) \geq 0$.

Most of the results we prove below hold for any such function f, but we will need to specialize to the function $k \mapsto 1/k$ to obtain our main theorem (and you will need to check that this does have the required properties).

We call $k \in \mathbb{N}$ *squarefree* if the only square that divides k is 1.

We introduce two basic finite sets of natural numbers.

```
def Nat.PrimesLt (n : ℕ) : Finset ℕ :=
    (range n).filter (fun p => Nat.Prime p)

def Nat.SqFreesLt (n : ℕ) : Finset ℕ:=
    (range n).filter (fun a => Squarefree a)
```

Lean knows that every $n \in \mathbb{N}$ can be expressed as a product $n = a * b^2$, with a squarefree and using this we can define the (noncomputable) decomposition $n \mapsto (a, b)$ as follows.

```
def Nat.decomp : ℕ → ℕ × ℕ :=
by
  intro n
  let a:= n.sq_mul_squarefree.choose
  let b:= n.sq_mul_squarefree.choose_spec.choose
  exact (a,b)
```

Lemma. *If $n \in \mathbb{N}$ and $n \geq 2$ then*

$$\log\left(\sum_{k \in \texttt{SqFreesLt}(n)} f(k)\right) \leq \sum_{p \in \texttt{PrimesLt}(n)} f(p).$$

Sketch: first use the fact that $\log x \leq y$ iff $x \leq \exp y$ (for $0 < x$).

If $k \in \texttt{SqFreesLt}(n)$ then k has no repeated prime factors and hence there exists a set $B_k \subseteq \texttt{PrimesLt}(n)$ such that

$$k = \prod_{p \in B_k} p,$$

given by factorising k. Moreover the map $k \mapsto B_k$ is injective hence

$$\sum_{k \in \texttt{SqFreesLt}(n)} f(k) \leq \sum_{B \subseteq \texttt{PrimesLt}(n)} f\left(\prod_{p \in B} p\right).$$

We can express the RHS as

$$\sum_{B \subseteq \texttt{PrimesLt}(n)} f\left(\prod_{p \in B} p\right) = \prod_{p \in \texttt{PrimesLt}(n)} (1 + f(p)).$$

Finally use the bound $1 + x \leq \exp x$ and take logarithms.

Using the decomposition `Nat.decomp` we can prove the other main intermediate result.

Lemma. *If $n \in \mathbb{N}$ then*

$$\sum_{k=0}^{n-1} f(k) \;\leq \left(\sum_{a \in \texttt{SqFreesLt}(n)} f(a)\right)\left(\sum_{b=0}^{n-1} f(b^2)\right).$$

Sketch: this follows from the decomposition and the fact that it is injective.

For the final result we set $f(n) = 1/n$ and prove the following simple bounds.

Lemma. *If $m \in \mathbb{N}$ then*

$$\frac{m}{2} + 1 \;\leq\; \sum_{k=0}^{2^m} \frac{1}{k}.$$

Lemma. *If $m \in \mathbb{N}$ then*

$$\sum_{k=0}^{m} \frac{1}{k^2} \leq 2.$$

The main result now follows via a calculation.

Possible extensions: 1. Improve the bound in the main theorem. 2. Make the function `decomp` computable.

6.7 A Thue equation (`ThueEquation`)

This is a more advanced topic in algebraic number theory.

A *Thue equation* is a Diophantine equation of the form

$$f(x, y) = c,$$

where $f \in \mathbb{Z}[x, y]$ is irreducible, homogeneous, and has degree at least 3. Each Thue equation has only finitely many solutions in integers.

In this project, consider the Thue equation

$$x^3 - 2y^3 = 1.$$

The aim of the project is to prove in lean that the only solutions in integers are $(1, 0)$ and $(-1, -1)$.

As a first step, we let $\alpha = \sqrt[3]{2}$. The equation can be rewritten as

$$N(x - y\alpha) = 1,$$

where N is the norm of an element of $\mathbb{Z}[\alpha]$. Therefore $x - y\alpha$ is a unit in $\mathbb{Z}[\alpha]$. One can prove that $u = 1 + \alpha + \alpha^2$ is a fundamental unit in $\mathbb{Z}[\alpha]$, which means that every unit is (up to a sign) a power of u, and every unit with norm 1 is a power of u. We then need to prove that the only powers u^n which have α^2-coefficient equal to zero are

$$u^0 = 1, \qquad u^{-1} = \alpha - 1.$$

These give the two solutions to the Thue equation

To do this in lean, you will need to begin by defining the ring $\mathbb{Z}[\alpha]$ together with its field embeddings in the complex numbers, and proving the properties of its norm. Proving that u is a fundamental unit involves some careful inequalities. The final step of finding which powers of n have zero α^2-coefficient can be done by congruences (let me know if you want to do this, so that I can show you the proof).